

## АННОТАЦИЯ

Выпускная квалификационная работа (далее ВКР) на тему: «Разработка программного комплекса на основе мультиагентной архитектуры с гибридной моделью интеграции для управления цифровым присутствием субъектов малого и среднего бизнеса».

**Объект исследования** — процесс управления цифровым присутствием субъектов малого и среднего бизнеса (МСБ) на внешних платформах.

**Предмет исследования** — автоматизация управления цифровым присутствием на основе мультиагентной архитектуры с гибридной моделью интеграции (программные интерфейсы приложений, API + роботизированная автоматизация процессов, RPA).

**Цель работы** — разработка и реализация программного комплекса на основе мультиагентной архитектуры с гибридной моделью интеграции, обеспечивающего автоматизацию управления цифровым присутствием субъектов МСБ на нескольких внешних площадках.

**Методы исследования:** сравнительный анализ существующих решений методом взвешенных критериев, объектно-ориентированное проектирование, моделирование бизнес-процессов (AS-IS / TO-BE), инфологическое и даталогическое моделирование данных, микросервисная декомпозиция, функциональное тестирование по критическому пути пользователя.

**Результаты работы.** Проведён анализ предметной области и семи существующих решений трёх классов (управление взаимоотношениями с клиентами, планирование публикаций, мониторинг репутации). Сформулированы функциональные и нефункциональные требования к системе. Спроектирована и реализована микросервисная система из шести компонентов и разделяемой библиотеки. Реализованы три платформенных агента (Telegram, VK, Яндекс.Бизнес), из которых два работают через официальные API платформ, а один — через браузерную автоматизацию (Playwright). Разработан

пользовательский интерфейс в виде одностраничного веб-приложения на базе Next.js 14 с 10 экранными формами.

**Объём работы:** 61 страница, 19 рисунков, 18 таблиц, 31 источник.

**Ключевые слова:** мультиагентная система, цифровое присутствие, малый и средний бизнес, оркестрация, браузерная автоматизация, роботизированная автоматизация процессов (RPA), микросервисная архитектура, большая языковая модель (LLM), Go, Next.js.

## СОДЕРЖАНИЕ

|                                                                                                         |    |
|---------------------------------------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ . . . . .                                                                                      | 5  |
| 1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ<br>ПРОЕКТНЫХ РЕШЕНИЙ . . . . .                                | 8  |
| 1.1 Описание предметной области и проблем управления цифровым<br>присутствием . . . . .                 | 8  |
| 1.1.1 Цифровое присутствие локального бизнеса: сущность и состав                                        | 8  |
| 1.1.2 Роль базовой бизнес-информации и проблема актуальности . .                                        | 9  |
| 1.1.3 Репутационный контур: отзывы и коммуникации . . . . .                                             | 10 |
| 1.1.4 Типовой сценарий (пример предметной области) . . . . .                                            | 10 |
| 1.1.5 Текущее состояние процесса управления цифровым<br>присутствием («как есть», AS-IS) . . . . .      | 11 |
| 1.2 Сравнительный анализ аналогов . . . . .                                                             | 12 |
| 1.2.1 Подход к анализу и критерии сравнения . . . . .                                                   | 12 |
| 1.2.2 Классы аналогов и примеры . . . . .                                                               | 13 |
| 1.2.3 Сравнительная таблица . . . . .                                                                   | 14 |
| 1.2.4 Выводы по аналогам и место разрабатываемой системы . . . .                                        | 16 |
| 1.3 Обзор мультиагентных систем, протоколов взаимодействия,<br>сравнение языков и фреймворков . . . . . | 17 |
| 1.3.1 Мультиагентные системы как технологическая основа . . . . .                                       | 17 |
| 1.3.2 Протоколы взаимодействия агентов и контекста модели . . . . .                                     | 18 |
| 1.3.3 Интеграции с внешними платформами через программные<br>интерфейсы . . . . .                       | 18 |
| 1.3.4 Браузерная автоматизация как метод интеграции . . . . .                                           | 19 |
| 1.3.5 Выбор технологического стека на основе метода взвешенных<br>критериев . . . . .                   | 21 |
| 1.4 Постановка задачи и требования к системе . . . . .                                                  | 22 |
| 1.4.1 Формализация проблемы . . . . .                                                                   | 22 |
| 1.4.2 Цель и задачи разработки . . . . .                                                                | 23 |

|       |                                                                      |    |
|-------|----------------------------------------------------------------------|----|
| 1.4.3 | Функциональные требования . . . . .                                  | 24 |
| 1.4.4 | Нефункциональные требования (рамка качества) . . . . .               | 25 |
| 1.4.5 | Требования к условиям эксплуатации и техническим средствам . . . . . | 26 |
| 1.4.6 | Концептуальная модель данных . . . . .                               | 27 |
| 1.4.7 | Целевое состояние процесса («будущее состояние», TO-BE) . . . . .    | 29 |
| 1.5   | Выводы по главе 1 . . . . .                                          | 30 |
| 2     | ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ . . . . .                                     | 32 |
| 2.1   | Проектирование архитектуры системы . . . . .                         | 32 |
| 2.1.1 | Протоколы межсервисного взаимодействия . . . . .                     | 34 |
| 2.1.2 | Диаграмма развёртывания . . . . .                                    | 36 |
| 2.1.3 | Маршрутизация запросов к LLM-провайдерам . . . . .                   | 38 |
| 2.1.4 | Протокол Agent-to-Agent (A2A) . . . . .                              | 38 |
| 2.1.5 | Алгоритм агентного цикла оркестратора . . . . .                      | 39 |
| 2.1.6 | Алгоритм ротации JWT-токенов . . . . .                               | 41 |
| 2.2   | Проектирование модели данных . . . . .                               | 43 |
| 2.2.1 | Инфологическая модель (ER-диаграмма) . . . . .                       | 43 |
| 2.2.2 | Даталогическая модель PostgreSQL . . . . .                           | 45 |
| 2.2.3 | Модель документов MongoDB . . . . .                                  | 47 |
| 2.2.4 | Типовые запросы к базам данных . . . . .                             | 48 |
| 2.3   | Проектирование пользовательского интерфейса . . . . .                | 48 |
| 2.3.1 | Структура навигации . . . . .                                        | 49 |
| 2.3.2 | Цветовое решение и типографика . . . . .                             | 50 |
| 2.3.3 | Экранные формы . . . . .                                             | 51 |
| 2.3.4 | Управление состоянием . . . . .                                      | 57 |
| 2.4   | Средства реализации . . . . .                                        | 58 |
| 2.5   | Инсталляция и настройка . . . . .                                    | 59 |
| 2.6   | Выводы по главе 2 . . . . .                                          | 59 |
|       | ЗАКЛЮЧЕНИЕ . . . . .                                                 | 61 |
|       | СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . . .                           | 63 |

## ВВЕДЕНИЕ

**Актуальность.** Управление цифровым присутствием является одной из ключевых задач для субъектов малого и среднего бизнеса (далее — МСБ) в условиях цифровой экономики. Информация о компании распределена по множеству платформ — геосервисам, социальным сетям, мессенджерам — каждая из которых обладает собственным интерфейсом управления. Актуализация данных (часов работы, адреса, контактов), мониторинг отзывов и публикация контента выполняются вручную на каждой площадке, что создаёт риск рассогласования данных и требует значительных трудозатрат.

Проблема усугубляется гетерогенностью интеграционных интерфейсов целевых платформ: одни предоставляют официальные программные интерфейсы приложений (API), тогда как другие — включая лидирующие на российском рынке геосервисы Яндекс.Бизнес (53 % пользователей) и 2ГИС (44 %) — не имеют публичных API для программного управления карточкой организации. Это делает невозможным создание универсального решения только на базе API и требует гибридного подхода, сочетающего работу через API с роботизированной автоматизацией процессов (RPA) для платформ без программного интерфейса.

Существующие решения — системы управления взаимоотношениями с клиентами (CRM, *customer relationship management*): Bitrix24, SendPulse, Kommo; инструменты планирования публикаций в социальных сетях (SMM, *social media management*): Hootsuite, Buffer, SMMplanner; платформы мониторинга репутации (ORM, *online reputation management*): YouScan — решают отдельные подзадачи, однако ни одно из них не обеспечивает комплексного управления консистентностью данных на разнородных платформах с единой оркестрацией действий специализированных агентов.

Мультиагентные системы (MAC) предоставляют архитектурную основу для решения указанной проблемы: специализированные агенты инкапсулируют метод интеграции с конкретной платформой, а центральный оркест-

ратор координирует их работу через единый протокол. Такой подход обеспечивает расширяемость — добавление новых платформ не требует модификации ядра системы.

**Объект исследования** — процесс управления цифровым присутствием субъектов малого и среднего бизнеса на внешних платформах.

**Предмет исследования** — автоматизация управления цифровым присутствием на основе мультиагентной архитектуры с гибридной моделью интеграции (API + RPA).

**Цель работы** — разработка программного комплекса на основе мультиагентной архитектуры с гибридной моделью интеграции для управления цифровым присутствием субъектов МСБ.

Для достижения указанной цели необходимо выполнить следующие задачи:

- 1) провести анализ предметной области и существующих решений;
- 2) сформулировать функциональные и нефункциональные требования к системе;
- 3) обосновать выбор технологий и архитектурных решений;
- 4) спроектировать архитектуру мультиагентной системы;
- 5) спроектировать модель данных;
- 6) реализовать прототип программного комплекса;
- 7) провести тестирование и оценку эффективности разработанного решения.

**Методы исследования:** сравнительный анализ существующих решений методом взвешенных критериев; объектно-ориентированное проектирование; моделирование бизнес-процессов (AS-IS / TO-BE); инфологическое и даталогическое моделирование данных; микросервисная декомпозиция; функциональное тестирование по критическому пути пользователя.

**Практическая значимость** работы состоит в создании программного комплекса, позволяющего субъектам МСБ управлять цифровым присутствием на нескольких площадках из единого интерфейса с использованием есте-

ственного языка. Система автоматизирует рутинные операции (обновление данных, публикация контента, мониторинг отзывов), снижает риск рассогласования информации между платформами и обеспечивает журналирование всех действий. Гибридная модель интеграции (API + RPA) позволяет работать в том числе с платформами, не предоставляющими публичного API, что является критичным для российского рынка геосервисов.

**Структура работы.** Работа состоит из введения, трёх глав, заключения, списка использованных источников и приложения.

В *первой главе* проведён анализ предметной области, выполнен сравнительный анализ существующих решений, обоснован выбор технологий и сформулированы требования к системе.

В *второй главе* описана архитектура системы, модель данных, проектирование пользовательского интерфейса и средства реализации.

В *третьей главе* приведены результаты тестирования и оценка эффективности разработанного решения.

# **1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ПРОЕКТНЫХ РЕШЕНИЙ**

## **1.1 Описание предметной области и проблем управления цифровым присутствием**

### **1.1.1 Цифровое присутствие локального бизнеса: сущность и состав**

Цифровое присутствие локального бизнеса можно определить как совокупность доступных в цифровых каналах данных и сигналов, по которым потенциальный клиент формирует ожидания о компании и принимает решение о взаимодействии. В практическом плане ядро цифрового присутствия составляют «карточки организации» (профили) на платформах бизнес-информации — сервисах, где отображаются адрес, часы работы, контакты и другие атрибуты компании [1, 2]. В экосистеме крупнейших поисковых платформ такие профили являются ключевыми точками входа: они отображаются в поиске и на картах, служат источником справочной информации для потребителя [3, 1].

Помимо профилей на картографических сервисах, компоненты цифрового присутствия включают:

- страницы и сообщества в социальных сетях (VK и др.);
- каналы и боты в мессенджерах (Telegram);
- агрегаторы товаров и услуг;
- отзывы и рейтинги на различных платформах.

Множественность каналов формирует задачу поддержания единообразной и актуальной информации о компании на каждом из них, что в терминологии данной работы обозначается как задача обеспечения *консистентности данных* цифрового присутствия.



Далее по тексту: **платформа** — внешний сервис (геосервис, социальная сеть, мессенджер и т. п.), через который бизнес представлен в сети; **канал** — конкретный тип присутствия или интеграции на платформе (профиль, сообщество, бот); **площадка** используется как близкий синоним *платформы* в контексте распределённости данных между разными сервисами.

### 1.1.2 Роль базовой бизнес-информации и проблема актуальности

Для потребителей критичны базовые сведения о компании — прежде всего часы работы и контактно-адресная информация. В Российской Федерации пользователи геосервисов часто используют онлайн-карты именно для уточнения графика работы (48 %) [4], что подчёркивает практическую значимость корректного расписания и контактных данных в карточке организации.

Международные опросные исследования (на примере США) показывают, что существенная доля аудитории отказывается от обращения в компанию при обнаружении некорректных данных онлайн [3, 2]. Это означает, что поддержание актуальности базовых атрибутов профиля (адрес, часы работы, контакты) является регулярной операционной задачей владельца бизнеса [1].

Отдельным фактором риска является рассогласование информации между площадками. По данным исследований, пользователи нередко видят конфликтующие сведения о компании на разных ресурсах [2]. На ряде платформ механизмы редактирования профилей предполагают внесение изменений не только владельцем, но и через предложения правок третьими лицами, что усиливает необходимость контроля и верификации данных [1].

В российском контексте ситуация дополнительно осложняется тем, что три геосервиса делят рынок: Яндекс.Карты используют 53 % пользователей, 2ГИС — 44 %, Google Карты — лишь 3 % [4]. При этом 63 % жителей России используют геосервисы для поиска офлайн-бизнеса [4]. Необходимость актуализации данных на нескольких площадках одновременно повышает трудоёмкость ручного управления и вероятность ошибок.

### **1.1.3 Репутационный контур: отзывы и коммуникации**

Помимо справочных данных, важной составляющей цифрового присутствия является репутационный контур: отзывы и реакция компании на них. В российском контексте при выборе мест в онлайн-картах пользователи обращают внимание на рейтинг (59 %) и отзывы (51 %) [4]. Следовательно, мониторинг отзывов и своевременная реакция на них должны рассматриваться как регулярный процесс, влияющий на доверие и конверсию.

Международные исследования демонстрируют, что ожидание ответа на отзывы распространено среди потребителей и наличие ответов коррелирует с выбором компании [5]. Это дополнительно поддерживает вывод о необходимости системной работы с отзывами [5, 3].

В российском сегменте онлайн-торговли исследования также отмечают влияние полноты информации о товаре или услуге и отзывов на доверие к продавцу [6].

Также в практических сценариях цифрового присутствия присутствуют коммуникации с клиентами в цифровых каналах (сообщения, вопросы), но в рамках данного раздела ключевой акцент делается на двух наиболее проверяемых контурах:

- 1) консистентность справочных данных;
- 2) управление отзывами как репутационным сигналом.

### **1.1.4 Типовой сценарий (пример предметной области)**

Для иллюстрации дальнейших разделов в качестве типового примера локального бизнеса рассматривается кофейня. По данным РОМИР, заведения общественного питания — наиболее частая категория поиска в офлайне (84 %), а геосервисы используют для уточнения графика работы (48 %) [4]. Для кофейни это делает критичными:

- корректные часы работы (чтобы клиент не пришёл в нерабочее время);
- адрес и маршрут;

— быстрые ответы на вопросы и отзывы (ассортимент, обслуживание, скорость приготовления и т. п.) [4, 5].

Кофейня представляет собой малое предприятие с ограниченным штатом, для которого выделение отдельного сотрудника на управление цифровым присутствием экономически нецелесообразно. Это делает задачу автоматизации особенно актуальной именно для данного сегмента бизнеса.

### **1.1.5 Текущее состояние процесса управления цифровым присутствием («как есть», AS-IS)**

Для формирования требований к разрабатываемой системе необходимо зафиксировать текущее состояние процесса управления цифровым присутствием субъектов малого и среднего бизнеса (МСБ) — модель AS-IS. На основании анализа предметной области выделены следующие характерные особенности текущего процесса:

- 1) владелец или сотрудник бизнеса вручную авторизуется на каждой платформе (Яндекс.Бизнес, 2ГИС, VK, Telegram) через отдельный веб-интерфейс;
- 2) при изменении бизнес-информации (часы работы, адрес, контакты) обновление выполняется последовательно на каждой площадке с индивидуальными формами редактирования;
- 3) мониторинг отзывов и сообщений требует поочерёдной проверки каждой платформы; единого потока событий не существует;
- 4) отсутствует «эталонный» источник данных — актуальная информация хранится только на самих площадках, что затрудняет обнаружение расхождений;
- 5) результаты действий не журналируются централизованно — невозможно отследить, кто и когда обновил данные на конкретной платформе.

Укрупнённая схема текущего процесса (AS-IS) представлена на рисунке 1. Основные недостатки процесса — высокая трудоёмкость ручных опе-

раций, риск рассогласования данных между площадками и отсутствие централизованного контроля — формируют предпосылки к проектированию автоматизированной системы.

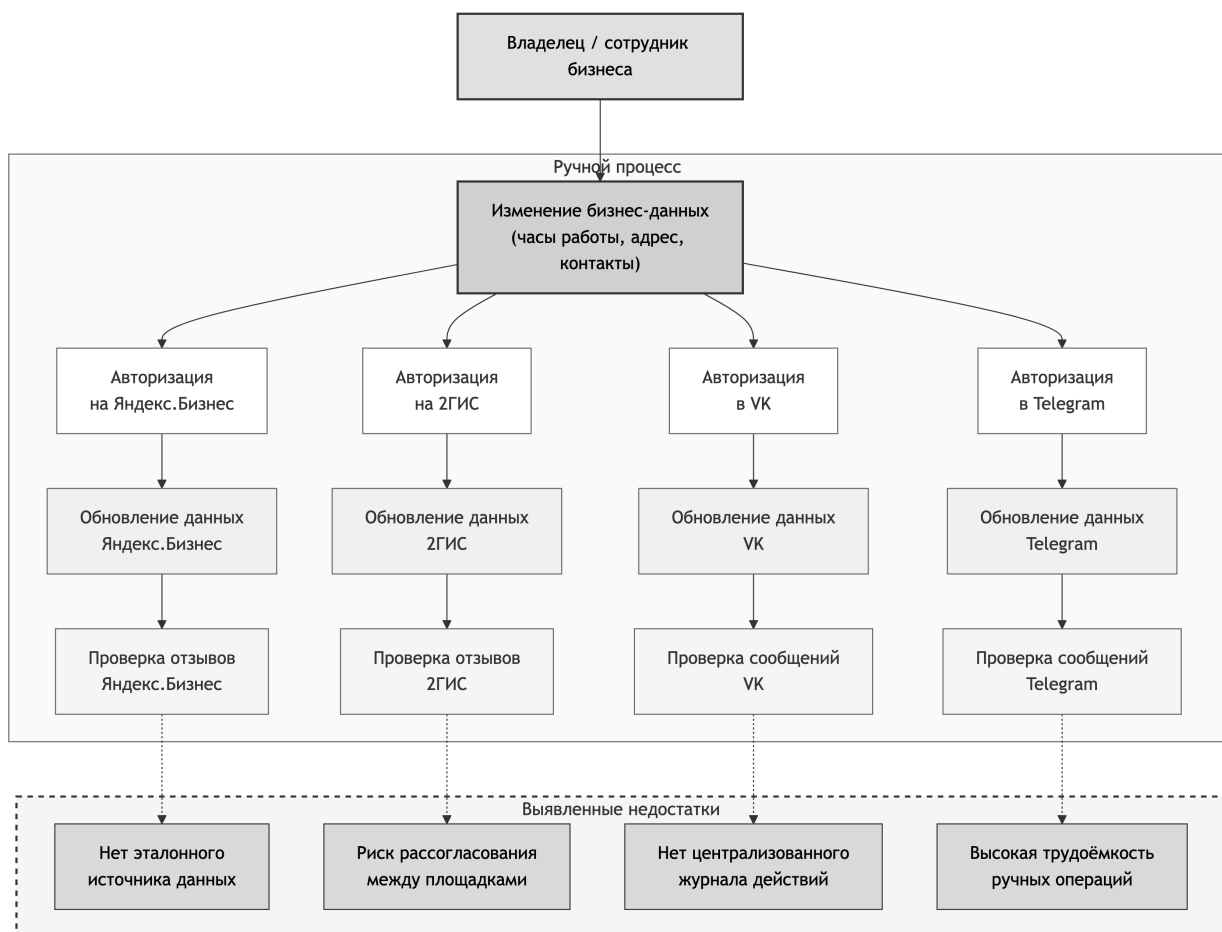


Рисунок 1 – Укрупнённая схема текущего процесса управления цифровым присутствием (AS-IS)

## 1.2 Сравнительный анализ аналогов

### 1.2.1 Подход к анализу и критерии сравнения

В рамках данного раздела рассматриваются аналоги, которые частично удовлетворяют потребности в инструментах управления цифровыми каналами МСБ. Перечень аналогов охватывает три основных класса решений: системы класса *customer relationship management* (CRM), инструменты планирования публикаций в социальных сетях (SMM) и решения для мониторинга репутации и упоминаний (ORM, *online reputation management*).

Для сравнения используется метод взвешенных критериев: каждому критерию присваивается удельный вес, отражающий его значимость для задачи управления цифровым присутствием МСБ, а каждый аналог оценивается по шкале от 0 до 3 баллов (0 — функция отсутствует, 1 — реализована минимально, 2 — реализована частично, 3 — реализована полноценно). Итоговый балл вычисляется как взвешенная сумма оценок.

Критерии сравнения и их обоснование:

- **К1. Функциональный охват** (вес 0,20): CRM и контакт-центр; планирование публикаций; мониторинг упоминаний и репутации;
- **К2. Мультиканальность** (вес 0,15): количество поддерживаемых каналов и степень «единого окна»;
- **К3. Автоматизация** (вес 0,15): наличие встроенных сценариев, правил, массовых операций;
- **К4. Управление консистентностью данных** (вес 0,25): поддержка «эталона» и контроль расхождений — ключевой критерий, определяющий специфику задачи;
- **К5. Трассируемость** (вес 0,10): журналирование действий и статусов;
- **К6. Расширяемость** (вес 0,15): наличие API и интеграций, возможность добавления новых каналов, в том числе для платформ без API.

Наибольший вес присвоен критерию К4, поскольку именно задача контроля консистентности данных на разнородных платформах является центральной в предметной области и не покрывается существующими решениями.

### 1.2.2 Классы аналогов и примеры

#### **CRM и омниканальные коммуникации.**

Bitrix24 описывает Contact Center как омниканальный модуль (формы, виджет, live chat, мессенджеры, телефония, электронная почта) [7, 8]. В спра-

вочной документации Bitrix24 отдельно описан механизм chat tracker для связывания переписок из разных каналов в одной CRM-сущности [9].

SendPulse описывает CRM как систему для управления коммуникациями и продажами в одном интерфейсе, включая коммуникации в мессенджерах [10]; в базе знаний приводится канбан-представление воронки сделок [11].

Kommo (ранее amoCRM) описывает Unified Inbox как единый интерфейс для коммуникаций (включая мессенджеры) и распределения диалогов между сотрудниками [12].

### **SMM-планирование и публикации.**

Hootsuite описывает планирование и публикацию контента из одного интерфейса; в справочной документации отмечается сценарий массовой загрузки публикаций с ограничением до 350 постов [13, 14].

Buffer описывает модуль Publish как инструмент планирования и публикации с подключением нескольких социальных сетей и возможностью настраивать посты под платформы [15, 16].

SMMplanner описывает автоматизированный постинг в социальные сети и мессенджеры; отдельно указывается поддержка постинга во VK и Telegram [17, 18].

### **Мониторинг упоминаний и ORM (social listening).**

YouScan описывает платформу мониторинга социальных медиа с визуальным анализом и модулем Visual Insights для распознавания логотипов и анализа изображений [19, 20].<sup>1</sup>

## **1.2.3 Сравнительная таблица**

Результаты балльной оценки аналогов по выбранным критериям приведены в таблице 1. Оценка выполнена по шкале 0–3 на основании анализа официальной документации и функциональных возможностей каждого ре-

---

<sup>1</sup>Количественные оценки эффективности (например, «до 80 % больше упоминаний») в материалах вендора следует трактовать как маркетинговые формулировки, а не как результаты независимого исследования [20].

шения. Оценки OneVoice являются проектными и отражают планируемые возможности разрабатываемой системы.

Таблица 1 – Балльная оценка аналогов по критериям сравнения

| Решение          | К1<br>0,20 | К2<br>0,15 | К3<br>0,15 | К4<br>0,25 | К5<br>0,10 | К6<br>0,15 | Итог        |
|------------------|------------|------------|------------|------------|------------|------------|-------------|
| Bitrix24         | 3          | 2          | 2          | 1          | 2          | 2          | 1,95        |
| SendPulse        | 2          | 2          | 2          | 0          | 1          | 2          | 1,40        |
| Kommo            | 2          | 2          | 2          | 0          | 1          | 2          | 1,40        |
| Hootsuite        | 1          | 2          | 2          | 0          | 1          | 2          | 1,20        |
| Buffer           | 1          | 2          | 2          | 0          | 1          | 2          | 1,20        |
| SMMplanner       | 1          | 1          | 2          | 0          | 0          | 1          | 0,80        |
| YouScan          | 1          | 1          | 1          | 0          | 1          | 2          | 0,90        |
| <b>OneVoice*</b> | <b>2</b>   | <b>2</b>   | <b>3</b>   | <b>3</b>   | <b>2</b>   | <b>3</b>   | <b>2,55</b> |

\* Проектные оценки OneVoice.

### 1.2.3.1 Обоснование балльных оценок

Суммарные баллы согласованы с весами критериев; ниже для каждого решения кратко указано, какие свойства продукта (по открытым материалам) соответствуют оценкам по К1–К6.

**Bitrix24** набирает наивысший суммарный балл среди аналогов (1,95). По К4 выставлен 1 балл: механизм chat tracker связывает переписки из разных каналов в одной CRM-сущности [9], что снижает фрагментацию коммуникаций, однако не заменяет отдельной функции «эталонного» профиля справочных полей и сквозного контроля расхождений между геосервисами. Остальные критерии отражают омниканальность Contact Center [7, 8], сценарии автоматизации и наличие API.

**SendPulse, Kommo** получают 0 по К4: фокус на коммуникациях и CRM без заявленного эталона справочных данных и без контроля расхождений между картами и мессенджерами [10, 12]. Баллы по К1–К3, К5, К6 следуют из описания каналов, воронок и интеграций [11, 10, 12].

**Hootsuite, Buffer, SMMplanner** — класс SMM: сильные стороны в планировании и публикациях [13, 14, 15, 16, 17, 18], по К4 — 0 (нет эталона и сверки справочных атрибутов между площадками).

**YouScan** — мониторинг упоминаний и визуальный анализ [19, 20]; по К4 — 0 (нет управления консистентностью карточек).

**OneVoice** (проектные оценки): максимум по К4 и К6 отражает целевую модель эталона и гибрид API+RPA. По К1 оценка 2: в MVP не входит полноценная аналитика репутации уровня ORM-класса. По К2 оценка 2: поддерживаются три ключевые платформы (VK, Telegram, Яндекс.Бизнес). По К5 оценка 2: в прототипе предусмотрены журналирование статусов задач, результатов агентов и ключевых событий по каналам; расширение до полного сквозного аудита всех действий запланировано на последующие итерации.

#### 1.2.4 Выводы по аналогам и место разрабатываемой системы

Количественный анализ показал, что рассмотренные решения решают отдельные подзадачи (коммуникации, публикации, мониторинг), но ни одно из них не набирает более 2,0 баллов из 3,0 возможных. Основной разрыв приходится на критерий К4 (управление консистентностью данных): все аналоги получили 0–1 балл. Ни одно из рассмотренных решений не обеспечивает:

- 1) поддержания «эталонных» данных о бизнесе и контроля расхождений между каналами как отдельной функции;
- 2) единого протокола оркестрации действий для платформ с различным уровнем открытости интерфейсов (API и RPA);
- 3) интеграции с ключевыми для российского рынка геосервисами (Яндекс.Бизнес, 2ГИС), не имеющими публичных API;
- 4) сквозного журналирования результатов по всем каналам.

OneVoice предполагает интеграцию перечисленных функций в единый процесс управления цифровым присутствием с оркестрацией действий специализированных агентов (проектный балл 2,55). Выявленные пробелы су-



ществующих решений формируют основу для определения требований к разрабатываемой системе и обоснования выбора технологий.

### **1.3 Обзор мультиагентных систем, протоколов взаимодействия, сравнение языков и фреймворков**

#### **1.3.1 Мультиагентные системы как технологическая основа**

Мультиагентная система (МАС) представляет собой архитектурный подход, при котором решение комплексной задачи достигается за счёт кооперации нескольких специализированных и относительно автономных агентов. Современные обзоры по кооперации в мультиагентных системах подчёркивают распределённость, специализацию и кооперацию как ключевые свойства такого класса систем [21].

Для задачи управления цифровым присутствием МСБ мультиагентный подход означает возможность разделить функции между агентами (сбор данных по каналам, проверка, подготовка действий, исполнение через интеграции) при наличии общего механизма координации. Основные преимущества мультиагентного подхода по сравнению с традиционной микросервисной архитектурой:

- 1) единый протокол взаимодействия вместо множества специализированных API, что радикально снижает связность компонентов;
- 2) автономность и проактивность агентов: агент может не только выполнить запрос, но и отклонить его с объяснением, предложить альтернативу или делегировать задачу;
- 3) инкапсуляция метода интеграции: оркестратор отправляет стандартизированное сообщение и не знает, использует ли агент REST API или браузерную автоматизацию;
- 4) возможность горизонтального расширения через добавление новых агентов без изменения ядра системы.

### 1.3.2 Протоколы взаимодействия агентов и контекста модели

Для снижения связности компонентов и повышения расширяемости системы целесообразно опираться на стандартизированные протоколы обмена сообщениями.

**A2A (Agent2Agent).** В спецификации A2A указано, что протокол предназначен для взаимодействия независимых агентных систем и опирается на HTTP, JSON-RPC 2.0 и Server-Sent Events (SSE) [22, 23]. Протокол определяет механизмы обнаружения возможностей агента, обмена задачами и получения результатов.

**MCP (Model Context Protocol).** В спецификации MCP описан стандартный протокол взаимодействия хостов, клиентов и серверов больших языковых моделей (LLM) с внешними источниками данных и инструментами на базе JSON-RPC 2.0 [24, 23]. MCP предоставляет унифицированный способ подключения инструментов и источников контекста к языковой модели.

В рамках OneVoice MCP может рассматриваться как базовый протокол «агент–инструменты» (доступ к источникам данных и инструментам), тогда как A2A — как протокол «агент–агент» при необходимости интероперабельности с внешними агентными системами.

### 1.3.3 Интеграции с внешними платформами через программные интерфейсы

Поскольку целевые цифровые каналы управляются внешними платформами, принципиальной предпосылкой реализации является наличие официальных API и корректная работа с авторизацией и ограничениями платформ.

Для целевых платформ, актуальных в российском контексте:

— **VK API** предоставляет официальные методы для работы с сообществами, публикациями и комментариями;

— **Telegram Bot API** предоставляет официальное HTTP-API для ботов [25];

— **Яндекс.Бизнес** и **2ГИС** — лидирующие геосервисы в России (53 % и 44 % пользователей соответственно) [4] — *не предоставляют публичного API* для программного управления карточкой организации.

Таким образом, гетерогенность интеграционных интерфейсов целевых платформ (наличие или отсутствие API) является ключевой проблемой, требующей гибридного подхода к проектированию агентов. Классификация платформ по типу интеграции представлена в таблице 2.

Таблица 2 – Классификация целевых платформ по типу интеграции

| Платформа               | Метод             | Приоритет | Примечание                                                 |
|-------------------------|-------------------|-----------|------------------------------------------------------------|
| VK                      | API (официальный) | MVP       | OAuth 2.0, методы для сообществ                            |
| Telegram                | API (Bot API)     | MVP       | Токен бота, HTTP-API                                       |
| Яндекс.Бизнес           | RPA (Playwright)  | MVP       | Нет публичного API, браузерная автоматизация               |
| 2ГИС                    | RPA (Playwright)  | Фаза 2    | Нет публичного API                                         |
| Google Business Profile | API (официальный) | Фаза 3    | 3 % использования в РФ, актуально для международных рынков |

#### 1.3.4 Браузерная автоматизация как метод интеграции

Для платформ, не предоставляющих публичного API (Яндекс.Бизнес, 2ГИС), применяется подход браузерной автоматизации (Robotic Process Automation (RPA)). Суть подхода — программная имитация действий пользователя в веб-интерфейсе платформы: навигация по страницам, заполнение форм, нажатие кнопок.

В научной литературе RPA рассматривается как подход к автоматизации повторяющихся и регламентированных задач в бизнес-процессах [26], а также отмечается устойчивый рост интереса к тематике и расширение области применения [27].

Среди инструментов браузерной автоматизации на 2025–2026 гг. выделяются три основных решения. Их сравнение по ключевым критериям приведено в таблице 3.

Таблица 3 – Сравнение инструментов браузерной автоматизации

| Критерий                              | Playwright                      | Selenium               | Puppeteer       |
|---------------------------------------|---------------------------------|------------------------|-----------------|
| Поддержка браузеров                   | Chromium, Firefox, WebKit       | Все основные           | Только Chromium |
| Языки и среды                         | Node.js, Python, Go, .NET, Java | Все основные языки     | Node.js         |
| Автоожидание элементов                | Встроено                        | Требует явных ожиданий | Частично        |
| Снижение риска детекции автоматизации | Через плагины                   | Через плагины          | Через плагины   |
| Режим headless (без GUI)              | Полноценный                     | Зависит от драйвера    | Полноценный     |

Для прототипа OneVoice выбран **Playwright** (Go-биндинг playwright-go) как инструмент с наилучшей поддержкой автоожидания элементов, режима *headless* (без графического интерфейса браузера) и кроссбраузерного тестирования.

Ключевые архитектурные решения для RPA-агентов:

- единый интерфейс агента (тот же протокол, что и у API-агентов) — оркестратор не знает, как именно агент взаимодействует с платформой;
- устойчивые селекторы (CSS/XPath с резервной стратегией) для устойчивости к изменениям DOM-структуры;
- механизм предварительной проверки (canary check) — проверка ключевых элементов страницы перед выполнением операции;
- сохранение снимков экрана при ошибке (screenshot-on-error) для диагностики сбоев;
- имитация человеческого поведения (случайные задержки 1–5 с между действиями) для снижения риска детекции.

### 1.3.5 Выбор технологического стека на основе метода взвешенных критериев

Для обоснования выбора серверного языка программирования применяется метод взвешенных критериев, аналогичный подходу, использованному при анализе аналогов (раздел 1.2). Критерии и их веса определены исходя из специфики задачи — реализации мультиагентной системы с конкурентной обработкой запросов к нескольким платформам:

— **Производительность и конкурентность** (вес 0,30): критичны для параллельного взаимодействия с несколькими платформами и обработки потоков событий;

— **Скорость прототипирования** (вес 0,25): определяет трудозатраты на реализацию в рамках ВКР;

— **Экосистема для API и интеграций** (вес 0,20): наличие библиотек для HTTP-клиентов, работы с JSON, браузерной автоматизации;

— **Простота развёртывания** (вес 0,15): размер артефактов, требования к среде выполнения;

— **Сопровождаемость кода** (вес 0,10): типизация, читаемость, инструменты анализа.

Результаты балльной оценки (шкала 0–3) представлены в таблице 4. Таблица 4 – Балльная оценка языков программирования для серверной части

| Критерий (вес)            | Go          | Python      | Java        |
|---------------------------|-------------|-------------|-------------|
| Производительность (0,30) | 3           | 1           | 3           |
| Прототипирование (0,25)   | 3           | 3           | 1           |
| Экосистема (0,20)         | 2           | 3           | 3           |
| Развёртывание (0,15)      | 3           | 2           | 1           |
| Сопровождаемость (0,10)   | 3           | 2           | 3           |
| <b>Взвешенный итог</b>    | <b>2,80</b> | <b>2,15</b> | <b>2,20</b> |

Go набирает наивысший балл (2,80) благодаря сочетанию высокой производительности (горутины обеспечивают конкурентность без накладных

расходов на потоки ОС), быстрого прототипирования (минималистичный синтаксис, быстрая компиляция) и простоты развёртывания (единый статический бинарный файл без зависимостей от среды выполнения).

Аналогичным образом для выбора фреймворка пользовательского интерфейса проведено сравнение, результаты которого представлены в таблице 5.

Таблица 5 – Балльная оценка фреймворков для клиентской части

| Критерий (вес)                | Next.js     | Nuxt.js     | Angular     |
|-------------------------------|-------------|-------------|-------------|
| SSR и маршрутизация (0,25)    | 3           | 3           | 2           |
| Скорость разработки (0,25)    | 3           | 3           | 2           |
| Экосистема компонентов (0,20) | 3           | 2           | 2           |
| Размер сообщества (0,15)      | 3           | 2           | 3           |
| Интеграция с REST/SSE (0,15)  | 3           | 2           | 2           |
| <b>Взвешенный итог</b>        | <b>3,00</b> | <b>2,50</b> | <b>2,15</b> |

На основании проведённого анализа для прототипа OneVoice выбран стек: **Go 1.24+** для серверной части и агентов; клиентская часть — **Next.js 15** на базе React 18 с Tailwind CSS и shadcn/ui; хранение данных — **PostgreSQL 16** и **MongoDB 7**. Next.js обеспечивает поддержку серверного рендеринга (SSR), нативную интеграцию с Server-Sent Events для потоковых ответов оркестратора и обширную экосистему готовых компонентов.

## 1.4 Постановка задачи и требования к системе

На основании анализа предметной области (раздел 1.1), выявленных пробелов существующих решений (раздел 1.2) и обзора доступных технологий (раздел 1.3) в настоящем разделе формулируются требования к разрабатываемой системе и описывается целевое состояние процесса.

### 1.4.1 Формализация проблемы

Задача автоматизации управления цифровым присутствием МСБ на нескольких внешних площадках (геосервисы и управление карточками орга-

низаций — Яндекс.Бизнес, 2ГИС; социальные сети — VK; мессенджеры — Telegram) включает поддержание актуальности данных и обработку обратной связи. Проблема формализуется следующим образом:

- данные о компании и коммуникации с клиентами распределены по нескольким платформам;
- обновления выполняются вручную или разрозненными инструментами, что повышает риск ошибок и несогласованности;
- отсутствие единого контура контроля и журналирования затрудняет выявление «истины» по бизнес-данным и управление качеством процессов.

Дополнительным фактором, усложняющим решение, является гетерогенность интеграционных интерфейсов целевых платформ: одни предоставляют официальные API (VK, Telegram), тогда как другие, включая лидирующие на российском рынке геосервисы (Яндекс.Бизнес, 2ГИС), не имеют публичных API для программного управления карточкой организации.

#### **1.4.2 Цель и задачи разработки**

Целью данной работы является разработка программного комплекса на основе мультиагентной архитектуры с гибридной моделью интеграции для управления цифровым присутствием субъектов МСБ.

Для достижения указанной цели в настоящем разделе решаются следующие задачи:

- определить функциональные и нефункциональные требования к системе;
- описать целевое состояние (TO-BE) процесса управления цифровым присутствием;
- предложить концептуальную модель данных (сущности и связи), обеспечивающую реализацию требований.

Методологически требования следует оформлять в соответствии с практиками инженерии требований и характеристиками «хороших требо-

ваний» (полнота, однозначность, проверяемость и др.), описанными в стандарте ISO/IEC/IEEE 29148 [28].

### 1.4.3 Функциональные требования

Ниже приведён перечень функциональных требований (далее — **ФТ**), составленный на основе анализа предметной области и выявленных пробелов существующих решений.

**ФТ-1. Управление бизнес-объектами:** создание и ведение карточки бизнеса (организация, филиал, локация), включая базовые атрибуты и метаданные.

**ФТ-2. Подключение внешних каналов:** привязка к бизнес-объекту внешних платформ и каналов (аккаунты, профили) с хранением параметров интеграции и статусов подключений.

**ФТ-3. Контроль консистентности данных:** хранение «эталонных» значений бизнес-атрибутов и их актуальных значений по каналам; обнаружение расхождений и формирование задач на синхронизацию или проверку.

**ФТ-4. Оркестрация действий:** постановка и исполнение задач по обновлению данных и контента через специализированных агентов; фиксация результата исполнения и ошибок.

**ФТ-5. Контур коммуникаций и обратной связи:** регистрация входящих событий (сообщения, отзывы, комментарии) и привязка к бизнес-объекту и каналу; поддержка статусов обработки и назначения ответственного.

**ФТ-6. Журналирование и трассируемость:** хранение истории изменений (кто, когда, что изменил), статусов задач и ключевых событий по каналам.

**ФТ-7. Роли пользователей и доступ:** разграничение прав с учётом принципа минимально необходимых привилегий. В системе предусмотрены три роли, функциональность которых описана в таблице 6.



Таблица 6 – Роли пользователей системы и их функциональность

| Роль                     | Функциональность                                                                                                                                                                                                                |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Владелец бизнеса (owner) | Полный доступ к бизнес-объекту: создание и удаление бизнеса, подключение и отключение внешних каналов, управление эталонными данными, назначение ролей сотрудникам, просмотр журнала действий и аналитики, управление подпиской |
| Администратор (admin)    | Управление каналами и контентом: редактирование эталонных данных, инициирование синхронизации, ответы на отзывы, публикация контента, просмотр аналитики. Без права управления ролями и подпиской                               |
| Оператор (member)        | Ограниченный доступ: просмотр входящих сообщений и отзывов, подготовка ответов, работа с задачами в рамках назначенных каналов. Без права изменения эталонных данных и настроек интеграций                                      |

#### 1.4.4 Нефункциональные требования (рамка качества)

Для структурирования нефункциональных требований (далее — **НФТ**) целесообразно использовать модель качества ISO/IEC 25010:2023; полный перечень характеристик и подхарактеристик приведён в стандарте и справочных материалах [29, 30]. Предварительный набор из восьми характеристик, согласованных с целями системы, в привязке к ISO/IEC 25010 представлен в таблице 7.

Таблица 7 – Нефункциональные требования к системе

| Характеристика                      | Требование                                                                                                                                                                                                   |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Функциональная пригодность          | Система покрывает заявленные сценарии: управление каналами, контроль консистентности, обработка обратной связи                                                                                               |
| Производительная эффективность      | Обработка типовых операций (плановые синхронизации, приём событий, формирование задач) выполняется в приемлемое время при росте числа каналов и сущностей                                                    |
| Совместимость и интероперабельность | Интеграции реализуются через официальные API или браузерную автоматизацию (RPA) и допускают расширение списка платформ; единый интерфейс агента обеспечивает прозрачность метода интеграции для оркестратора |
| Удобство взаимодействия             | Интерфейс поддерживает типовые роли и минимизирует ошибки пользователя при массовых обновлениях                                                                                                              |
| Надёжность                          | Система сохраняет целостность данных и историю действий при частичных сбоях интеграций                                                                                                                       |
| Информационная безопасность         | Защищённое хранение токенов и секретов, аудит действий и разграничение доступа                                                                                                                               |
| Сопровождаемость                    | Возможность добавлять новые агенты (как API-, так и RPA-агенты) и изменять правила оркестрации без полной переработки системы                                                                                |
| Переносимость                       | Возможность развёртывания в целевой инфраструктуре (контейнеризация, типовая установка)                                                                                                                      |

#### 1.4.5 Требования к условиям эксплуатации и техническим средствам

Помимо характеристик качества, для обеспечения полноты требований необходимо зафиксировать условия эксплуатации и требования к техническим средствам.

**Требования к условиям эксплуатации.** Система предназначена для работы в режиме 24/7 в серверном окружении с доступом в Интернет. Пользовательский интерфейс доступен через современные веб-браузеры (последние две стабильные версии основных браузеров). Специальных требований к рабочему месту пользователя не предъявляется.

**Требования к составу и параметрам технических средств.** Минимальные требования к серверной инфраструктуре для развёртывания прототипа: процессор с 2 ядрами, 4 ГБ оперативной памяти, 20 ГБ дискового пространства (SSD). Для RPA-агентов, использующих браузерную автоматизацию, требуется дополнительно не менее 1 ГБ оперативной памяти на каждый экземпляр браузера в режиме *headless*.

**Требования к информационной и программной совместимости.** Серверная часть должна функционировать в среде контейнеризации и поддерживать оркестрацию контейнеров для промышленного развёртывания. Система должна использовать реляционную и документную СУБД для хранения структурированных и слабоструктурированных данных соответственно. Клиентская часть должна корректно работать в современных веб-браузерах. Конкретный состав и версии программных компонентов определяются в техническом задании (Приложение А).

**Требования к программной документации.** В состав документации должны входить: описание REST API (формат OpenAPI/Swagger), инструкция по развёртыванию системы, руководство пользователя. Техническое задание оформляется в соответствии с ГОСТ 34.602-2020 и выносится в Приложение А к настоящей работе.

#### **1.4.6 Концептуальная модель данных**

На основании сформулированных требований определены основные группы сущностей концептуальной модели данных: бизнес-объекты и их атрибуты, пользователи и роли, каналы и профили на внешних платформах, эталонные и фактические данные (для контроля консистентности), события и коммуникации, задачи и их статусы, журнал действий. Аналоги CRM-класса демонстрируют практику консолидации переписок из разных каналов в одной CRM-сущности [9, 10]. Также распространённым представлением «во-

ронки» является канбан-доска со стадиями и карточками, перемещаемыми между стадиями [11, 31].

Концептуальная ER-диаграмма, отображающая связи между основными сущностями, представлена на рисунке 2. Детализация полей, описание конкретных сущностей и даталогическая модель приводятся в главе 2.

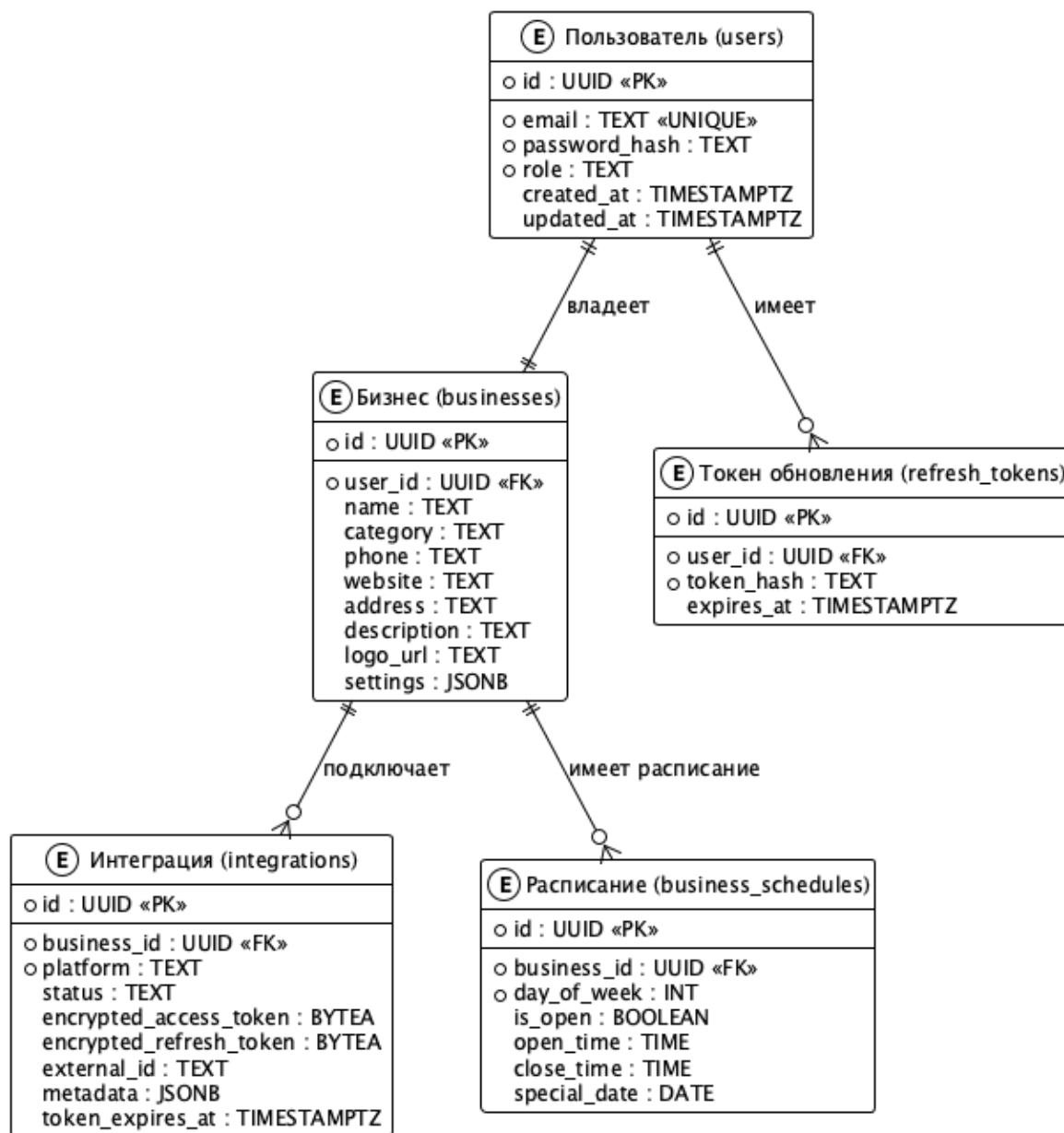


Рисунок 2 – Концептуальная ER-диаграмма инфологической модели данных

### 1.4.7 Целевое состояние процесса («будущее состояние», TO-BE)

В целевом состоянии (TO-BE) управление цифровым присутствием строится как управляемый процесс: пользователь задаёт «эталон» и правила, система фиксирует расхождения и события, далее агенты выполняют автоматизированные действия, а результаты журналируются. Укрупнённая схема целевого процесса представлена на рисунке 3.

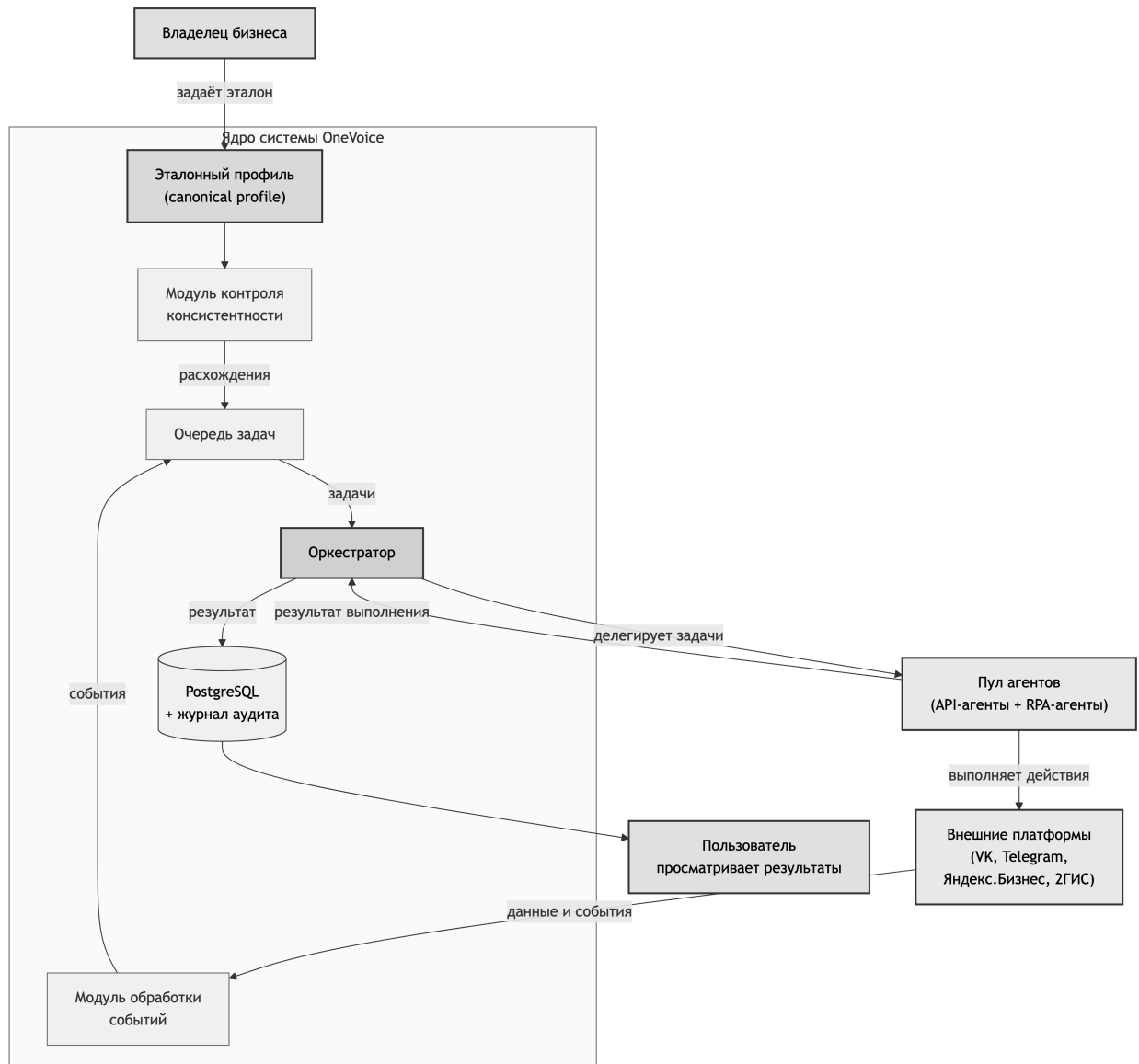


Рисунок 3 – Укрупнённая схема целевого процесса управления цифровым присутствием (TO-BE)

Ключевой особенностью целевого процесса является гибридная модель интеграции: оркестратор делегирует задачи специализированным аген-

там, каждый из которых инкапсулирует метод взаимодействия с конкретной платформой. Схема взаимодействия агентов с внешними платформами представлена на рисунке 4.

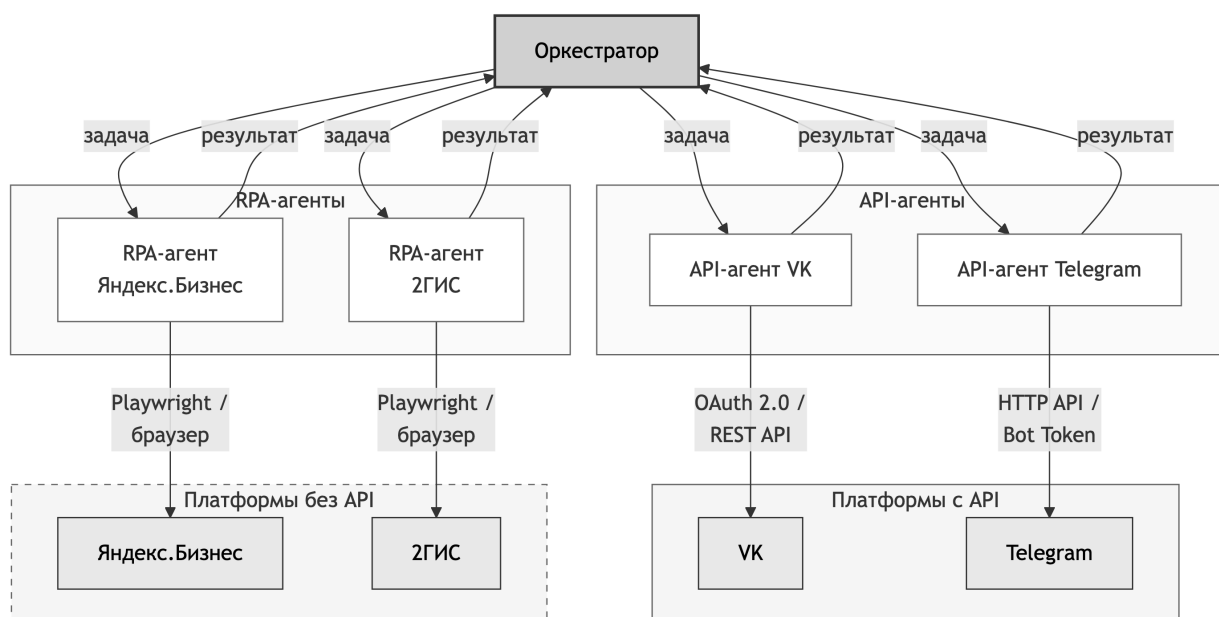


Рисунок 4 – Схема взаимодействия агентов с внешними платформами

На рисунке 4 API-агенты (VK, Telegram) взаимодействуют с платформами через официальные API, тогда как RPA-агенты (Яндекс.Бизнес, 2ГИС) используют браузерную автоматизацию (Playwright). При этом оркестратор оперирует единым протоколом задач и результатов, не зная о методе интеграции конкретного агента.

Указанные схемы являются укрупнёнными и служат основанием для дальнейшего проектирования архитектуры (глава 2) и планирования тестирования и оценки эффективности (глава 3).

## 1.5 Выводы по главе 1

В первой главе выполнен анализ предметной области управления цифровым присутствием МСБ и показано, что ключевыми контурами являются консистентность справочных данных и управление репутацией. Построение модели AS-IS выявило основные ограничения текущего процесса: высокую

трудоёмкость ручных операций, риск рассогласования данных между площадками и отсутствие централизованного контроля.

Сравнительный анализ аналогов (CRM, SMM, ORM) методом взвешенных критериев подтвердил, что представленные на рынке решения не решают задачу комплексного управления консистентностью данных в разнородных каналах.

В качестве архитектурной основы обоснована мультиагентная модель с гибридной интеграцией API + RPA, а также выбран технологический стек реализации. По результатам анализа сформулированы функциональные и нефункциональные требования, определены роли пользователей, предложена концептуальная модель данных и описано целевое состояние ТО-ВЕ. Полученные результаты служат основанием для проектирования архитектуры и реализации системы в следующей главе.

## 2 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ

### 2.1 Проектирование архитектуры системы

На основании требований, сформированных в первой главе, была спроектирована архитектура системы OneVoice. Ключевым архитектурным решением стало применение микросервисного подхода, обусловленного следующими факторами:

- 1) необходимость независимого масштабирования отдельных компонентов (платформенных агентов);
- 2) изоляция сбоев — отказ одного агента не влияет на работу остальных;
- 3) возможность использования различных технологий для разных компонентов (Go для серверной части, Node.js для клиентского приложения, Playwright для агента браузерной автоматизации);
- 4) упрощение добавления новых платформ без модификации существующего кода.

Система состоит из шести микросервисов и разделяемой библиотеки. Общая архитектура представлена на диаграмме компонентов (рисунок 5).



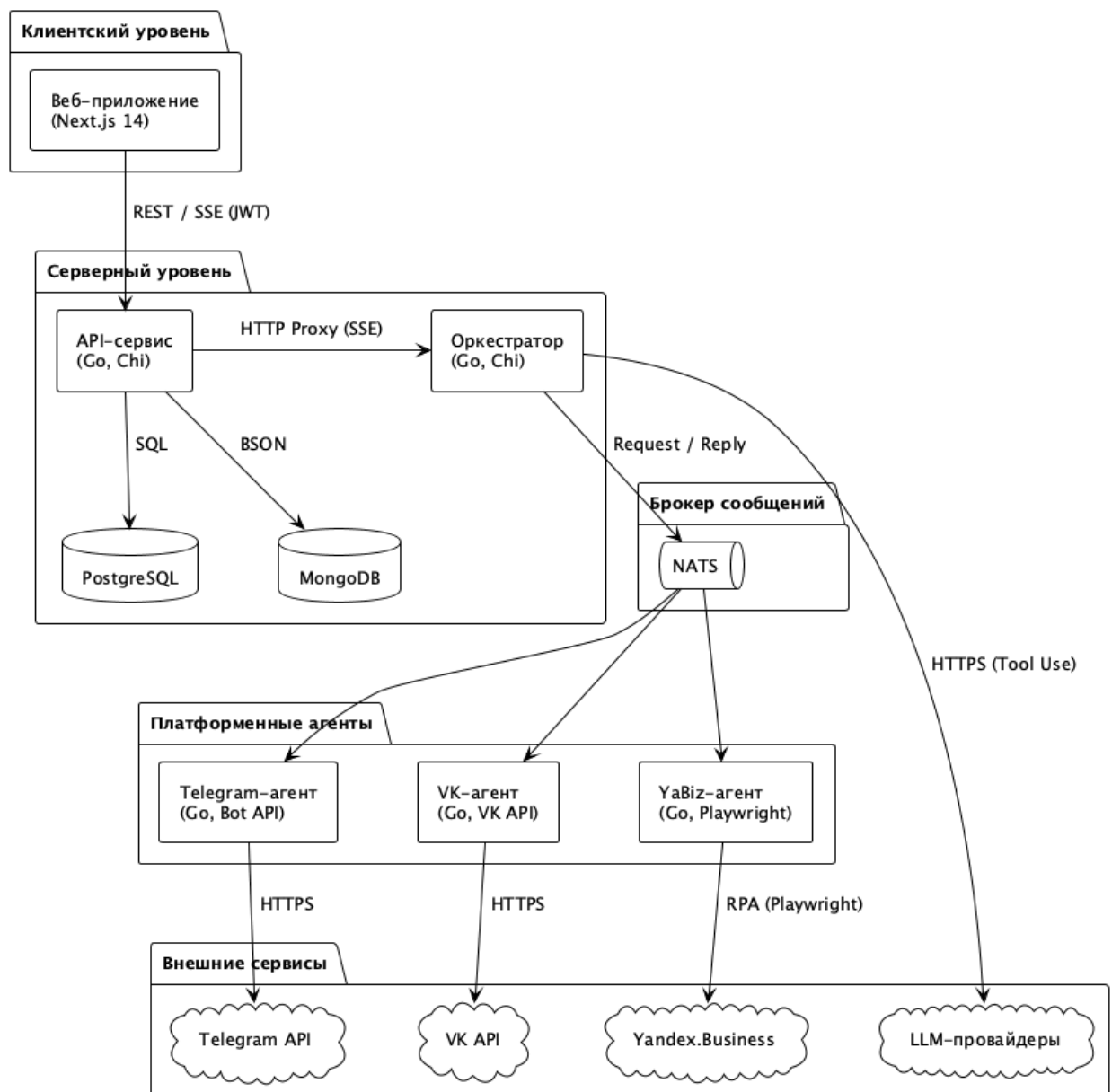


Рисунок 5 – Диаграмма компонентов системы OneVoice

Состав и назначение микросервисов приведены в таблице 8.

Таблица 8 – Микросервисы системы OneVoice

| Модуль                 | Назначение                                                                       | Технологии                    |
|------------------------|----------------------------------------------------------------------------------|-------------------------------|
| API-сервис             | REST API, аутентификация, операции над бизнес-данными (CRUD), проксирование чата | Go, Chi, PostgreSQL, MongoDB  |
| Оркестратор            | Агентный цикл LLM, диспетчеризация инструментов через NATS                       | Go, Chi, NATS                 |
| Веб-приложение         | Панель управления (панель управления)                                            | Next.js 14, React, TypeScript |
| Telegram-агент         | Интеграция с Telegram Bot API                                                    | Go, Telegram Bot API          |
| VK-агент               | Интеграция с VK API                                                              | Go, VK API                    |
| Yandex.Business-агент  | Интеграция с Яндекс.Бизнес через браузерную автоматизацию                        | Go, Playwright                |
| Общая библиотека (pkg) | Доменные модели, маршрутизатор LLM-запросов, программный каркас протокола A2A    | Go                            |

### 2.1.1 Протоколы межсервисного взаимодействия

В системе используются три протокола взаимодействия, характеристики которых приведены в таблице 9.

Таблица 9 – Протоколы взаимодействия между компонентами системы

| Протокол         | Направление                 | Паттерн                | Назначение                                             |
|------------------|-----------------------------|------------------------|--------------------------------------------------------|
| REST (HTTP/JSON) | Веб-приложение → API-сервис | Запрос/ответ           | CRUD-операции, аутентификация                          |
| SSE              | API-сервис → Веб-приложение | Однонаправленный поток | Потоковая передача ответов LLM и статусов инструментов |
| NATS             | Оркестратор → Агенты        | Запрос/ответ           | Диспетчеризация задач платформенным агентам            |

Взаимодействие между компонентами при основных сценариях использования представлено на диаграмме последовательности (рисунок 6).

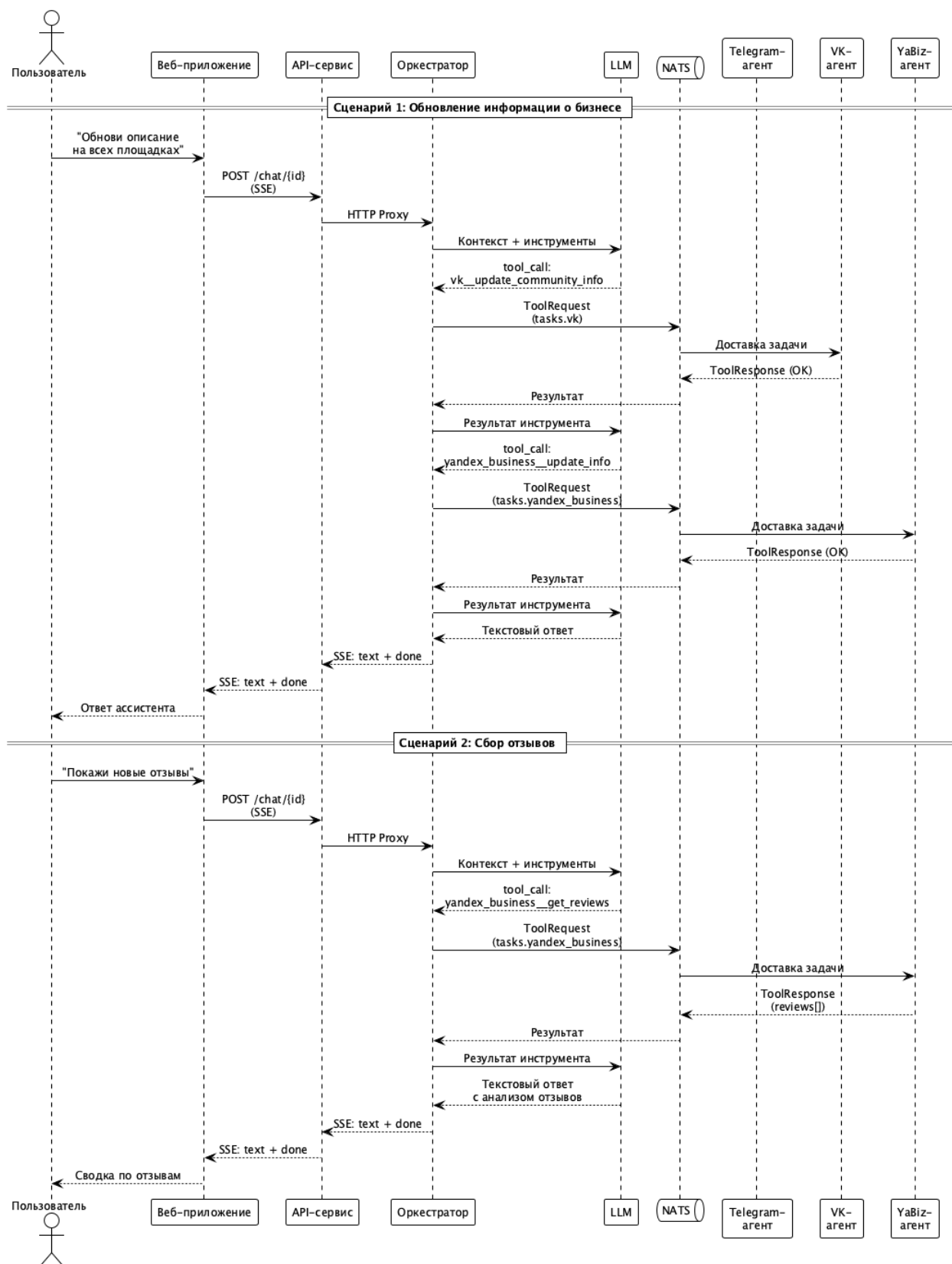


Рисунок 6 – Диаграмма последовательности:  
обновление информации и сбор отзывов

**Сценарий 1: обновление информации о бизнесе.** Пользователь вводит в чат запрос на обновление описания бизнеса на всех площадках. Че-

рез API-сервис сообщение проксируется к оркестратору, который формирует контекст (профиль бизнеса, список активных интеграций) и отправляет запрос к LLM с описанием доступных инструментов. LLM последовательно определяет необходимые инструменты (`vk__update_community_info`, `yandex_business__update_info`) и возвращает вызовы инструментов. Оркестратор маршрутизирует каждую задачу через NATS соответствующему агенту, который выполняет действие на платформе. После получения результатов всех вызовов LLM формирует текстовый ответ.

**Сценарий 2: сбор отзывов.** Пользователь запрашивает новые отзывы. LLM вызывает инструмент `yandex_business__get_reviews`, RPA-агент собирает отзывы с платформы и возвращает их оркестратору. LLM анализирует полученные данные и формирует сводку для пользователя. Весь процесс транслируется клиенту в реальном времени через SSE.

### 2.1.2 Диаграмма развёртывания

Все компоненты системы контейнеризованы с использованием Docker и управляются через Docker Compose. Диаграмма развёртывания представлена на рисунке 7.

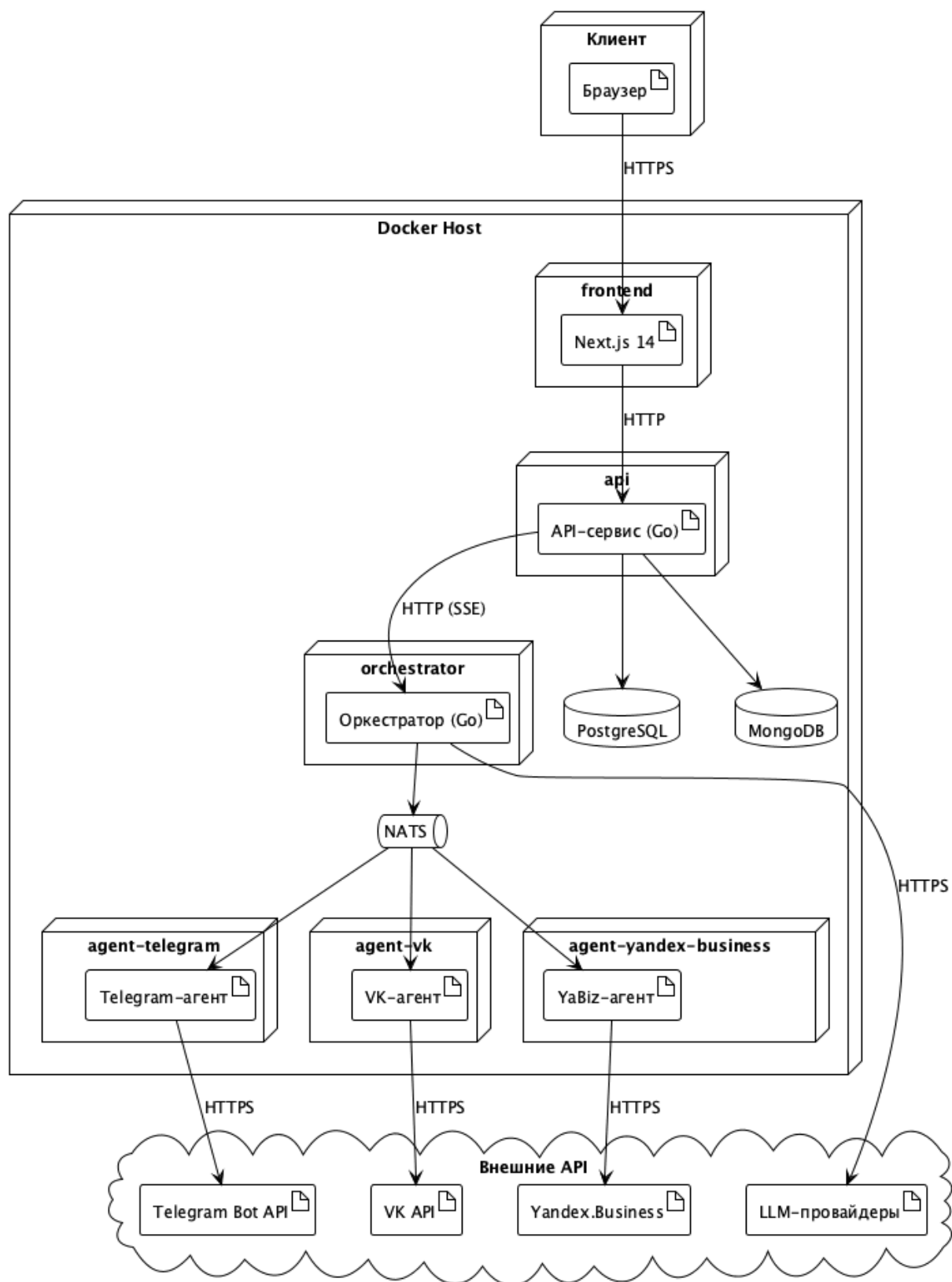


Рисунок 7 – Диаграмма развёртывания системы OneVoice

Инфраструктурные компоненты включают: PostgreSQL для хранения реляционных данных, MongoDB для хранения документов и NATS в качестве брокера сообщений.

### 2.1.3 Маршрутизация запросов к LLM-провайдерам

Для взаимодействия с большими языковыми моделями реализован LLM-маршрутизатор, поддерживающий несколько провайдеров: OpenRouter (основной, мультимодельный), OpenAI (прямое подключение), Anthropic (прямое подключение) и Self-Hosted (OpenAI-совместимые конечные точки). Маршрутизатор обеспечивает единый интерфейс для оркестратора независимо от используемого провайдера, что позволяет переключаться между моделями без изменения логики приложения.

### 2.1.4 Протокол Agent-to-Agent (A2A)

Для стандартизации взаимодействия между оркестратором и платформенными агентами разработан протокол A2A, определяющий две структуры данных:

- 1) ToolRequest — запрос от оркестратора к агенту, содержащий идентификатор задачи, имя инструмента, аргументы и идентификатор бизнеса;
- 2) ToolResponse — ответ агента, содержащий статус выполнения, результат или описание ошибки.

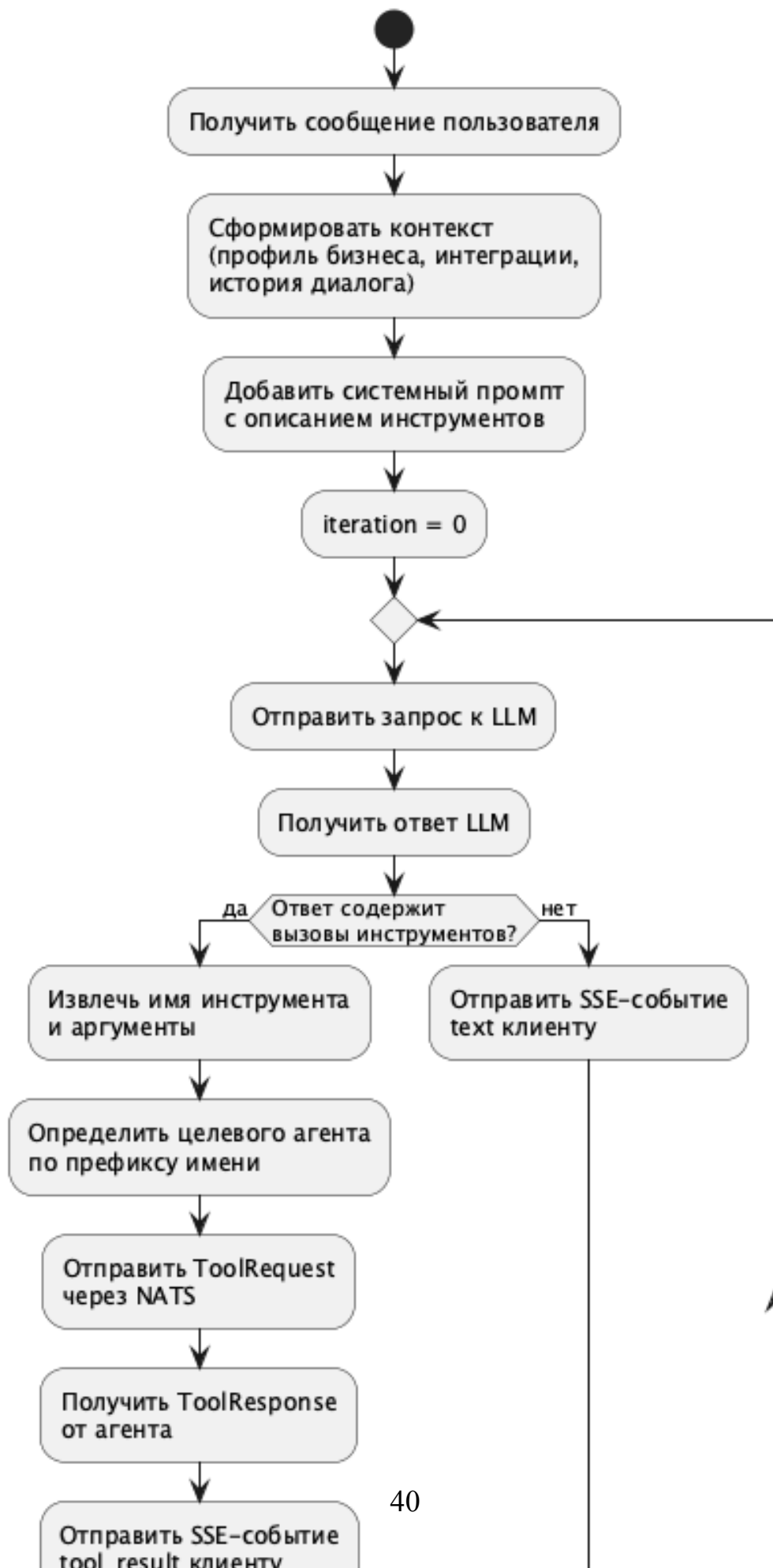
|          |    |                               |         |                        |
|----------|----|-------------------------------|---------|------------------------|
| Именован | ие | инструментов                  | следует | конвен                 |
| ции      |    | { платформа }__ { действие }, |         | например:              |
|          |    | telegram__send_channel_post,  |         | vk__publish_wall_post, |
|          |    | yandex_business__update_info. |         |                        |

Такой подход позволяет оркестратору автоматически определять целевого агента по префиксу имени инструмента и маршрутизировать задачу через NATS на соответствующий субъект (`tasks.telegram`, `tasks.vk`, `tasks.yandex_business`).

### **2.1.5 Алгоритм агентного цикла оркестратора**

Центральным элементом системы является агентный цикл оркестратора, реализующий итеративное взаимодействие с LLM и платформенными агентами. Алгоритм представлен на рисунке 8.

## Алгоритм агентного цикла оркестратора





Алгоритм работает следующим образом:

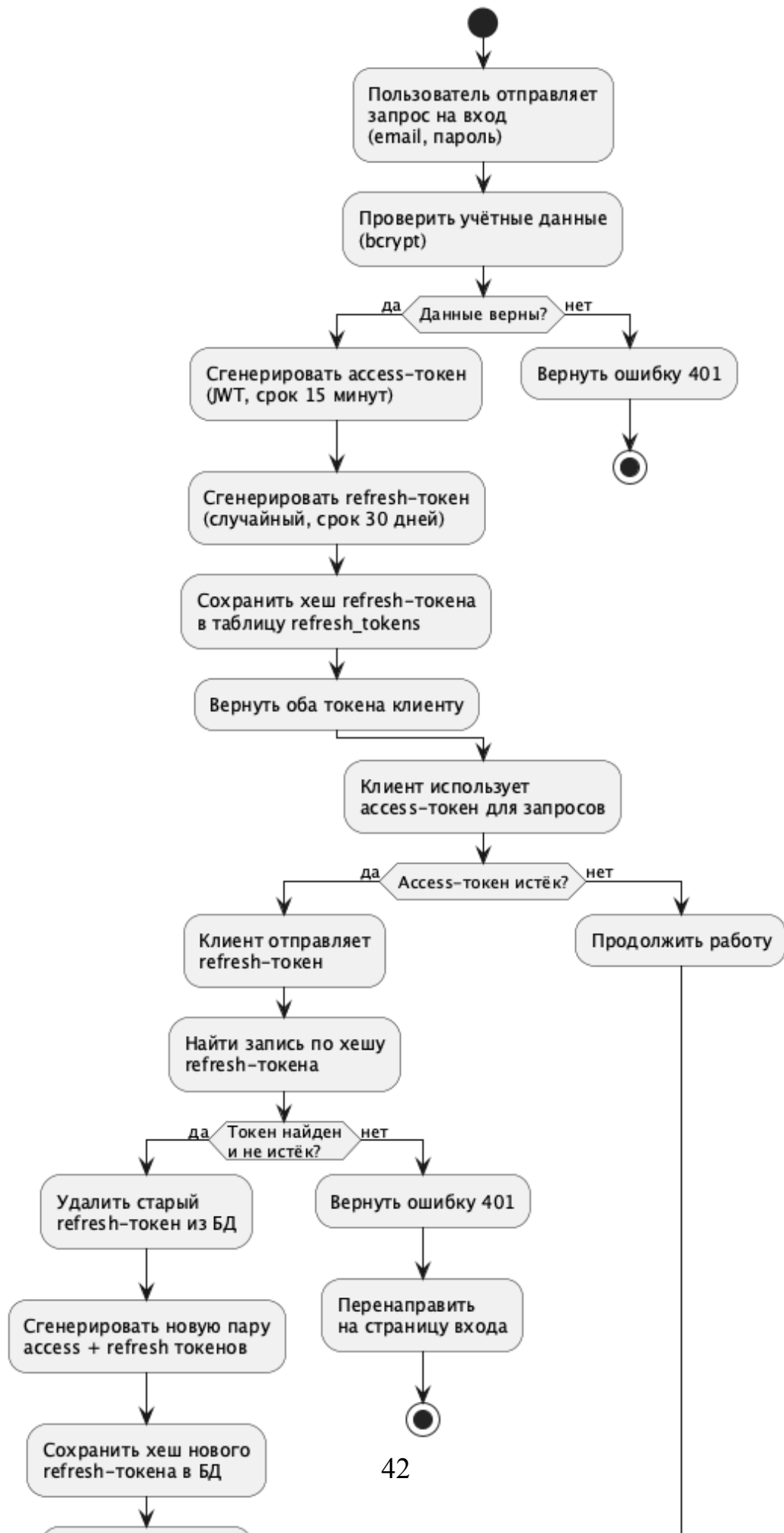
- 1) При получении сообщения пользователя оркестратор формирует контекст, включающий профиль бизнеса, список активных интеграций и историю диалога.
- 2) Контекст дополняется системной инструкцией (system prompt) с описанием доступных инструментов и отправляется к LLM-провайдеру.
- 3) LLM анализирует запрос и возвращает либо текстовый ответ, либо набор вызовов инструментов (tool calls).
- 4) Если ответ содержит вызовы инструментов, оркестратор извлекает имя инструмента и аргументы, определяет целевого агента по префиксу имени и отправляет запрос `ToolRequest` через NATS.
- 5) Результат выполнения (`ToolResponse`) добавляется в контекст диалога, и цикл повторяется.
- 6) Цикл завершается, когда LLM возвращает текстовый ответ без вызовов инструментов или при достижении максимального числа итераций (`MAX_ITERATIONS = 10`).

Каждое промежуточное состояние (вызов инструмента, результат, текст) транслируется клиенту в реальном времени через SSE-события, обеспечивая прозрачность процесса для пользователя.

### **2.1.6 Алгоритм ротации JWT-токенов**

Для обеспечения безопасной аутентификации в системе реализован механизм ротации JWT-токенов. Алгоритм представлен на рисунке 9.

## Алгоритм ротации JWT-токенов



Механизм аутентификации основан на паре токенов:

1) **Access-токен** — JWT со сроком действия 15 минут, содержащий идентификатор пользователя и роль. Используется для авторизации каждого API-запроса.

2) **Refresh-токен** — случайная строка со сроком действия 30 дней. Хранится в базе данных в виде хеша (SHA-256) и используется для обновления пары токенов.

При обновлении токенов применяется ротация: старый refresh-токен удаляется из базы данных, а новый сохраняется. Это предотвращает повторное использование скомпрометированных токенов. При выходе пользователя из системы refresh-токен отзывается, что делает невозможным дальнейшее обновление сессии.

## 2.2 Проектирование модели данных

При проектировании модели данных использован подход гетерогенного хранения данных (polyglot persistence): реляционная СУБД PostgreSQL применяется для структурированных данных с ACID-гарантиями (пользователи, бизнесы, интеграции), а документоориентированная СУБД MongoDB — для данных с гибкой схемой (диалоги, сообщения, публикации, отзывы, задачи агентов).

### 2.2.1 Инфологическая модель (ER-диаграмма)

Инфологическая модель реляционной части системы представлена на рисунке 10.

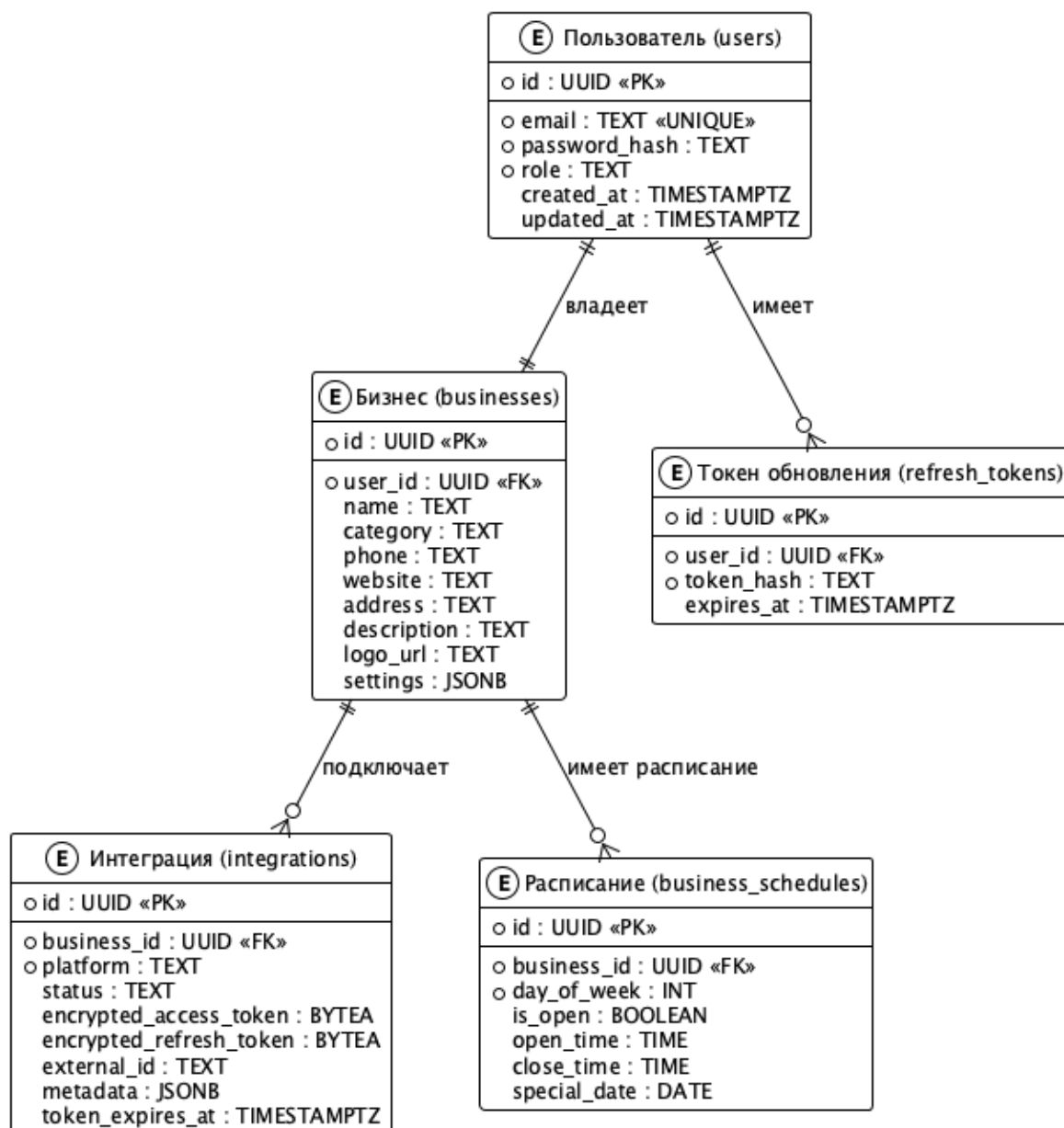


Рисунок 10 – ER-диаграмма реляционной модели данных

Модель включает следующие сущности:

- 1) **Пользователь (User)** — учётная запись с аутентификационными данными и ролью в системе;
- 2) **Бизнес (Business)** — профиль бизнеса, связанный с пользователем отношением «один ко многим»;
- 3) **Интеграция (Integration)** — подключение к внешней платформе с зашифрованными токенами доступа (AES-256-GCM);
- 4) **Расписание (BusinessSchedule)** — график работы бизнеса с поддержкой особых дат;

5) **Токен обновления (RefreshToken)** — хешированный токен для ротации JWT.

### 2.2.2 Даталогическая модель PostgreSQL

Структура основных таблиц реляционной базы данных приведена в таблицах 10–14.

Таблица 10 – Структура таблицы users

| Поле          | Тип           | Описание                                   |
|---------------|---------------|--------------------------------------------|
| id            | UUID          | Первичный ключ, генерируется автоматически |
| email         | TEXT (UNIQUE) | Электронная почта пользователя             |
| password_hash | TEXT          | Хеш пароля (bcrypt)                        |
| role          | TEXT          | Роль: owner, admin, member                 |
| created_at    | TIMESTAMPTZ   | Дата создания записи                       |
| updated_at    | TIMESTAMPTZ   | Дата последнего обновления                 |

Таблица 11 – Структура таблицы businesses

| Поле        | Тип       | Описание                       |
|-------------|-----------|--------------------------------|
| id          | UUID      | Первичный ключ                 |
| user_id     | UUID (FK) | Ссылка на владельца (users.id) |
| name        | TEXT      | Название бизнеса               |
| category    | TEXT      | Категория деятельности         |
| phone       | TEXT      | Контактный телефон             |
| website     | TEXT      | Адрес веб-сайта                |
| address     | TEXT      | Физический адрес               |
| description | TEXT      | Краткое описание               |
| logo_url    | TEXT      | URL логотипа                   |
| settings    | JSONB     | Дополнительные настройки       |

Таблица 12 – Структура таблицы integrations

| Поле                    | Тип         | Описание                                  |
|-------------------------|-------------|-------------------------------------------|
| id                      | UUID        | Первичный ключ                            |
| business_id             | UUID (FK)   | Ссылка на бизнес                          |
| platform                | TEXT        | Платформа: telegram, vk, yandex_business  |
| status                  | TEXT        | Статус: pending, active, error            |
| encrypted_access_token  | BYTEA       | Токен доступа (AES-256-GCM)               |
| encrypted_refresh_token | BYTEA       | Токен обновления (AES-256-GCM)            |
| external_id             | TEXT        | Внешний идентификатор (ID канала, группы) |
| metadata                | JSONB       | Метаданные подключения                    |
| token_expires_at        | TIMESTAMPTZ | Срок действия токена                      |

Ограничение уникальности UNIQUE (business\_id, platform, external\_id) позволяет подключать несколько каналов одной платформы к одному бизнесу, что важно для работы с несколькими Telegram-каналами или VK-сообществами.

Таблица 13 – Структура таблицы business\_schedules

| Поле         | Тип       | Описание                                                                      |
|--------------|-----------|-------------------------------------------------------------------------------|
| id           | UUID      | Первичный ключ                                                                |
| business_id  | UUID (FK) | Ссылка на бизнес (businesses.id)                                              |
| day_of_week  | INTEGER   | День недели (0 — воскресенье, 6 — суббота)                                    |
| is_open      | BOOLEAN   | Признак рабочего дня                                                          |
| open_time    | TEXT      | Время открытия (формат HH:MM)                                                 |
| close_time   | TEXT      | Время закрытия (формат HH:MM)                                                 |
| special_date | DATE      | Особая дата (если задана, запись описывает исключение из обычного расписания) |

Таблица 14 – Структура таблицы refresh\_tokens

| Поле       | Тип         | Описание                          |
|------------|-------------|-----------------------------------|
| id         | UUID        | Первичный ключ                    |
| user_id    | UUID (FK)   | Ссылка на пользователя (users.id) |
| token_hash | TEXT        | Хеш refresh-токена (SHA-256)      |
| expires_at | TIMESTAMPTZ | Срок действия токена              |
| created_at | TIMESTAMPTZ | Дата создания записи              |

### 2.2.3 Модель документов MongoDB

Для хранения данных с гибкой структурой используется MongoDB. Схема коллекций представлена на рисунке 11.

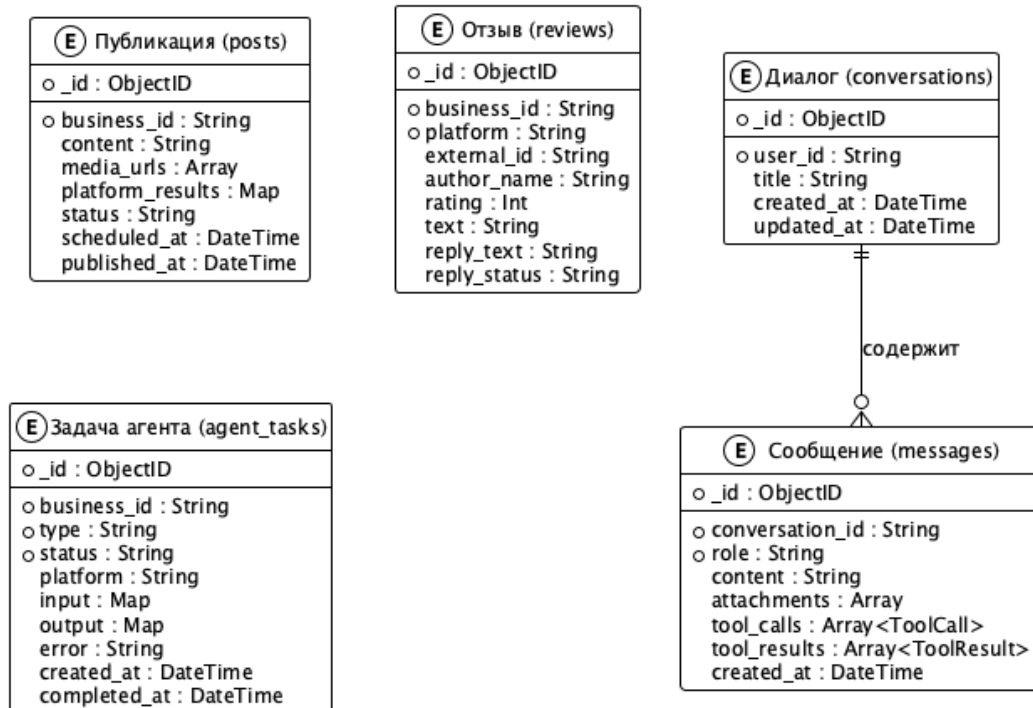


Рисунок 11 – Схема коллекций MongoDB

Основные коллекции:

- 1) **conversations** — диалоги пользователя с ИИ-ассистентом;
- 2) **messages** — сообщения в диалогах с поддержкой вложений, вызовов инструментов и их результатов;
- 3) **posts** — публикации на платформах с результатами по каждой платформе;
- 4) **reviews** — отзывы клиентов с различных платформ;
- 5) **agent\_tasks** — журнал задач, выполняемых платформенными агентами.

Выбор MongoDB для этих коллекций обусловлен необходимостью хранения вложенных объектов переменной структуры (массивы вызовов инструментов в сообщениях, результаты публикаций по платформам) без необходимости нормализации.

## 2.2.4 Типовые запросы к базам данных

В таблице 15 приведены типовые запросы, используемые в системе.

Таблица 15 – Типовые запросы к базам данных

| Операция                    | СУБД  | Описание                                                                      |
|-----------------------------|-------|-------------------------------------------------------------------------------|
| Аутентификация              | PG    | Поиск пользователя по email, проверка хеша пароля                             |
| Получение токена интеграции | PG    | Поиск активной интеграции по business_id и platform с расшифровкой токена     |
| Загрузка истории чата       | Mongo | Получение сообщений по conversation_id с вложенными tool_calls и tool_results |
| Фильтрация публикаций       | Mongo | Поиск по business_id с фильтрами по статусу и платформе, сортировка по дате   |
| Фильтрация отзывов          | Mongo | Поиск по business_id с фильтрами по платформе и статусу ответа                |
| Журнал задач агентов        | Mongo | Поиск по business_id с фильтрами по статусу и платформе                       |

## 2.3 Проектирование пользовательского интерфейса

Пользовательский интерфейс системы OneVoice реализован как одностраничное веб-приложение (SPA) на базе Next.js 14 с использованием компонентной библиотеки shadcn/ui. При проектировании интерфейса учитывались следующие принципы:

- 1) **Единообразие** — все страницы используют общий макет с боковой навигационной панелью;
- 2) **Адаптивность** — интерфейс корректно отображается на настольных компьютерах, планшетах и мобильных устройствах;
- 3) **Обратная связь** — действия пользователя сопровождаются визуальными индикаторами (статусы подключений, уведомления, анимации загрузки);
- 4) **Минимализм** — интерфейс не перегружен элементами, основной акцент на функциональности.



Выбор архитектуры одностраничного приложения (SPA) обусловлен спецификой основного сценария использования — диалога с ИИ-ассистентом в реальном времени. SPA позволяет поддерживать постоянное SSE-соединение для потоковой передачи ответов LLM без перезагрузки страницы, а также обеспечивает мгновенные переходы между разделами панели управления с сохранением состояния чата.

В качестве основного цвета выбран оттенок Indigo (#6366F1), обеспечивающий профессиональное восприятие интерфейса и высокий контраст с белым фоном (коэффициент контрастности 4.6:1 по WCAG 2.1 AA). Данный цвет является стандартным для библиотеки shadcn/ui, что гарантирует визуальную согласованность всех компонентов.

Шрифтовое оформление построено на гарнитуре Inter — свободном шрифте, оптимизированном для экранного отображения и поддерживающем кириллицу. Inter входит в стандартный набор системных шрифтов Next.js, что исключает необходимость загрузки дополнительных веб-шрифтов и ускоряет начальную отрисовку страницы.

### **2.3.1 Структура навигации**

Система разделена на две зоны: публичную (аутентификация) и защищённую (основной панель управления). Экранные формы системы перечислены в таблице 16.

Таблица 16 – Экранные формы системы OneVoice

| Страница    | Назначение              | Ключевые элементы                                                      |
|-------------|-------------------------|------------------------------------------------------------------------|
| Вход        | Аутентификация          | Форма: email, пароль, кнопка входа                                     |
| Регистрация | Создание аккаунта       | Форма: имя, email, пароль, подтверждение                               |
| Чат         | Диалог с ИИ-ассистентом | Список диалогов, окно сообщений, панель инструментов, быстрые действия |
| Интеграции  | Подключение платформ    | Карточки платформ, модальные окна подключения, статусы                 |
| Бизнес      | Профиль и расписание    | Форма профиля, редактор расписания по дням, особые даты                |
| Отзывы      | Управление отзывами     | Таблица с фильтрами, встроенный диалог ответа                          |
| Посты       | Журнал публикаций       | Таблица с фильтрами, раскрываемые строки с результатами по платформам  |
| Задачи      | Журнал задач агентов    | Таблица с фильтрами по статусу и платформе                             |
| Настройки   | Параметры аккаунта      | Информация о пользователе, смена пароля                                |

Навигация по защищённой зоне осуществляется через боковую панель шириной 240 пикселей, содержащую ссылки на все разделы и индикаторы подключения платформ в нижней части. На мобильных устройствах панель скрывается и вызывается жестом или кнопкой.

### 2.3.2 Цветовое решение и типографика

Цветовая палитра и параметры типографики приведены в таблице 17.

Таблица 17 – Цветовая палитра и типографика интерфейса

| Параметр                   | Значение            | Применение                                  |
|----------------------------|---------------------|---------------------------------------------|
| Основной цвет<br>(Primary) | #6366F1 (Indigo)    | Кнопки, активные элементы навигации, ссылки |
| Фон боковой панели         | #1E293B (Slate 800) | Навигационная панель                        |
| Фон контентной области     | #F8FAFC (Slate 50)  | Основная область страницы                   |
| Цвет успеха                | #22C55E (Green)     | Статусы «Подключено», «Опубликовано»        |
| Цвет ошибки                | #EF4444 (Red)       | Ошибки, статусы «Отключено»                 |
| Шрифт                      | Inter (системный)   | Основной текст, заголовки                   |
| Размер базового текста     | 14px                | Основной текст интерфейса                   |

### 2.3.3 Экранные формы

#### 2.3.3.1 Аутентификация

Страницы входа и регистрации выполнены в минималистичном стиле с центрированной формой на светлом фоне (рисунки 12, 13).

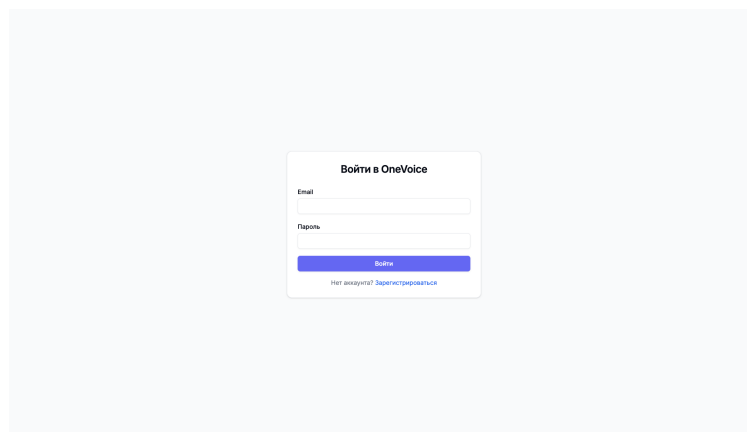


Рисунок 12 – Экранная форма авторизации

Рисунок 13 – Экранная форма регистрации

### 2.3.3.2 Профиль бизнеса и расписание

Страница профиля бизнеса содержит форму с основной информацией (название, категория, телефон, сайт, адрес, описание) и редактор расписания работы по дням недели с поддержкой выходных дней и особых дат (рисунки 14, 15).

Рисунок 14 – Экранная форма профиля бизнеса с расписанием

OneVoice  
ulitest2@test.com

Чат  
Интеграции  
**Бизнес**  
Отзывы  
Посты  
Задачи  
Настройки

ПЛАТФОРМЫ  
● Telegram  
● ВКонтакте  
● Яндекс.Бизнес  
Выйти

Описание  
Тестовый бизнес  
Сохранить

Расписание работы

Понедельник ☒ Открыто 09 : 00 — 21 : 00

Вторник ☒ Открыто 09 : 00 — 21 : 00

Среда ☒ Открыто 09 : 00 — 21 : 00

Четверг ☒ Открыто 09 : 00 — 21 : 00

Пятница ☒ Открыто 09 : 00 — 21 : 00

Суббота ☒ Открыто 09 : 00 — 21 : 00

Воскресенье ☐ Выходной

Особые даты  
Праздники и особый режим работы  
Нет особых дат  
Добавить дату

Сохранить расписание

Рисунок 15 – Редактор расписания работы

### 2.3.3.3 Управление интеграциями

Страница интеграций отображает карточки доступных платформ (Telegram, ВКонтакте, Яндекс.Бизнес) с индикаторами статуса подключения. Для каждой платформы предусмотрено модальное окно подключения с пошаговой процедурой (рисунок 16).

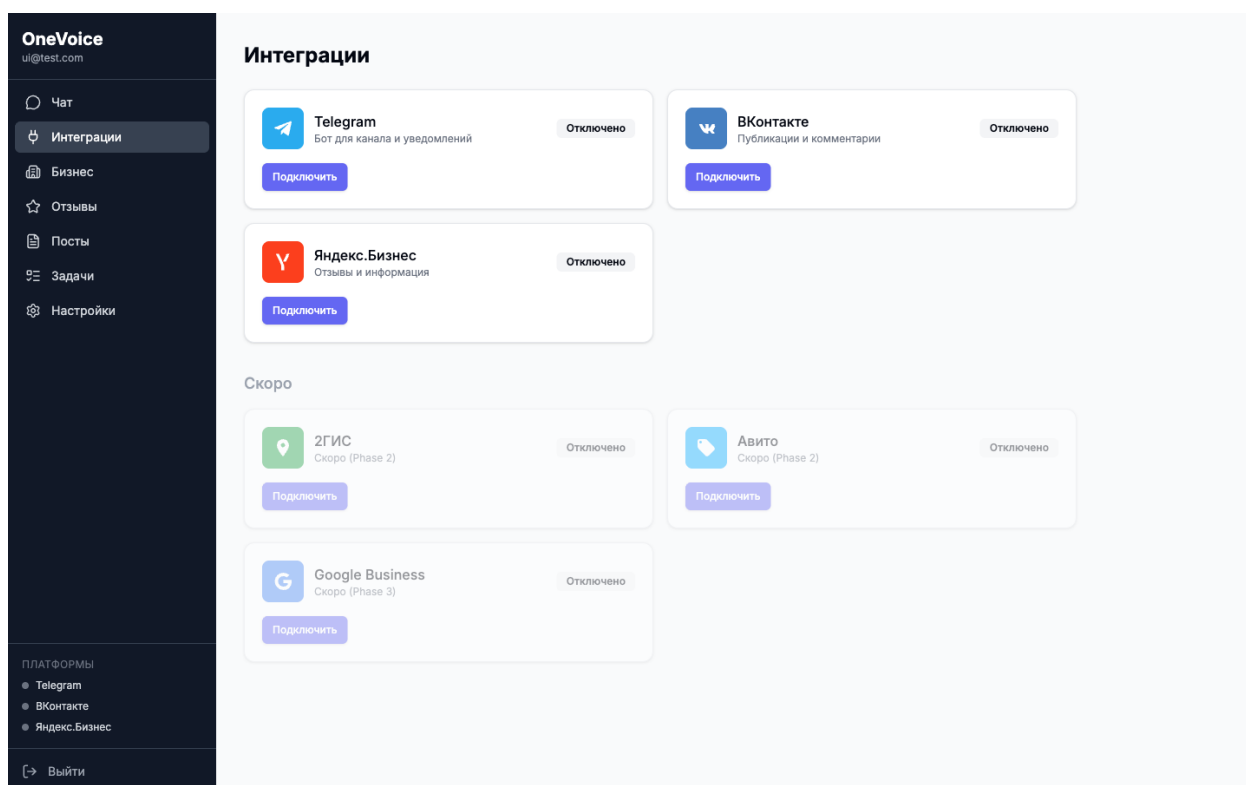


Рисунок 16 – Экранная форма управления интеграциями

#### 2.3.3.4 Журнал публикаций

Страница публикаций содержит таблицу с фильтрацией по платформе и статусу (все, опубликованы, запланированы, ошибки). Каждая строка раскрывается для отображения полного текста и результатов публикации по платформам (рисунки 17, 18).

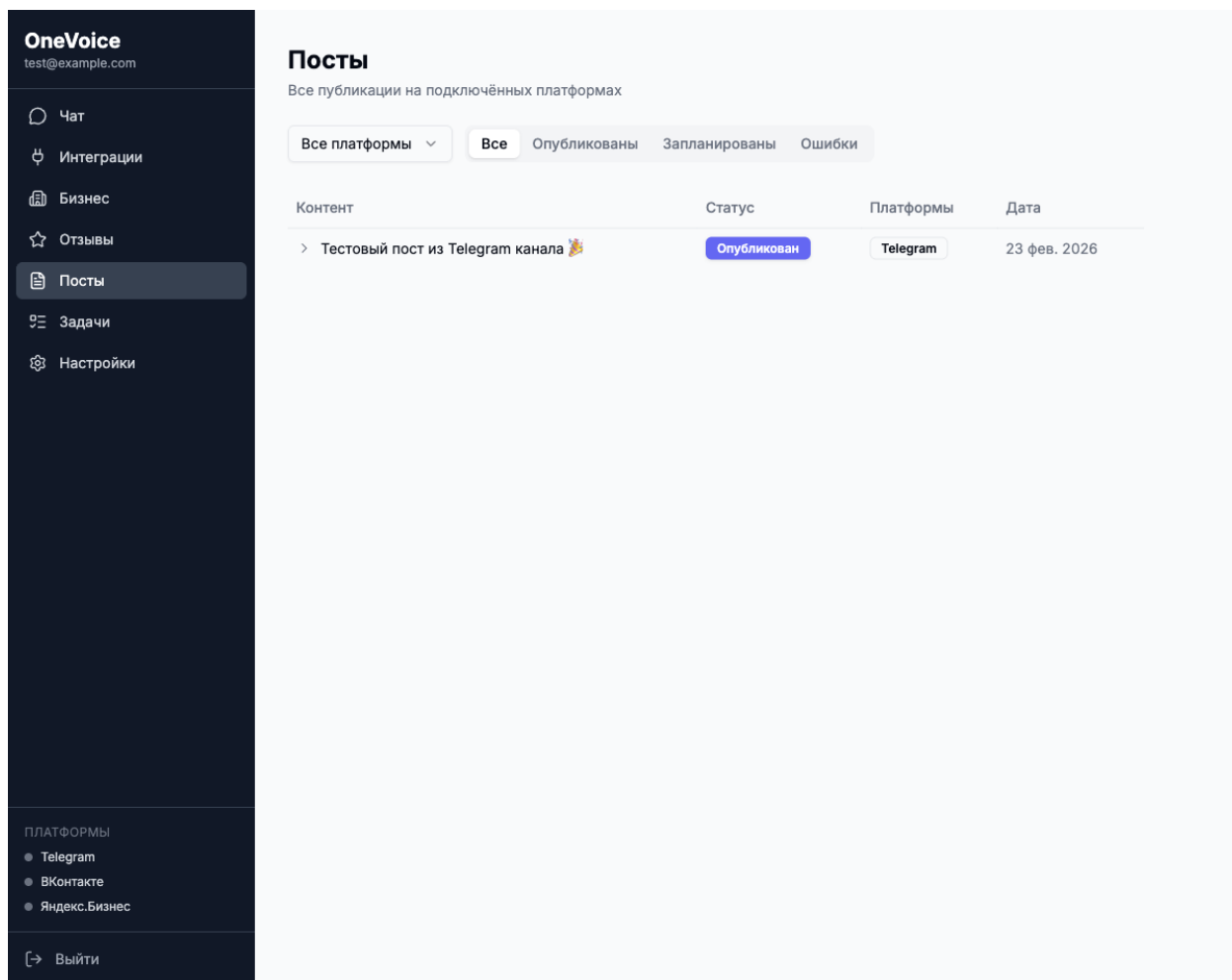


Рисунок 17 – Журнал публикаций

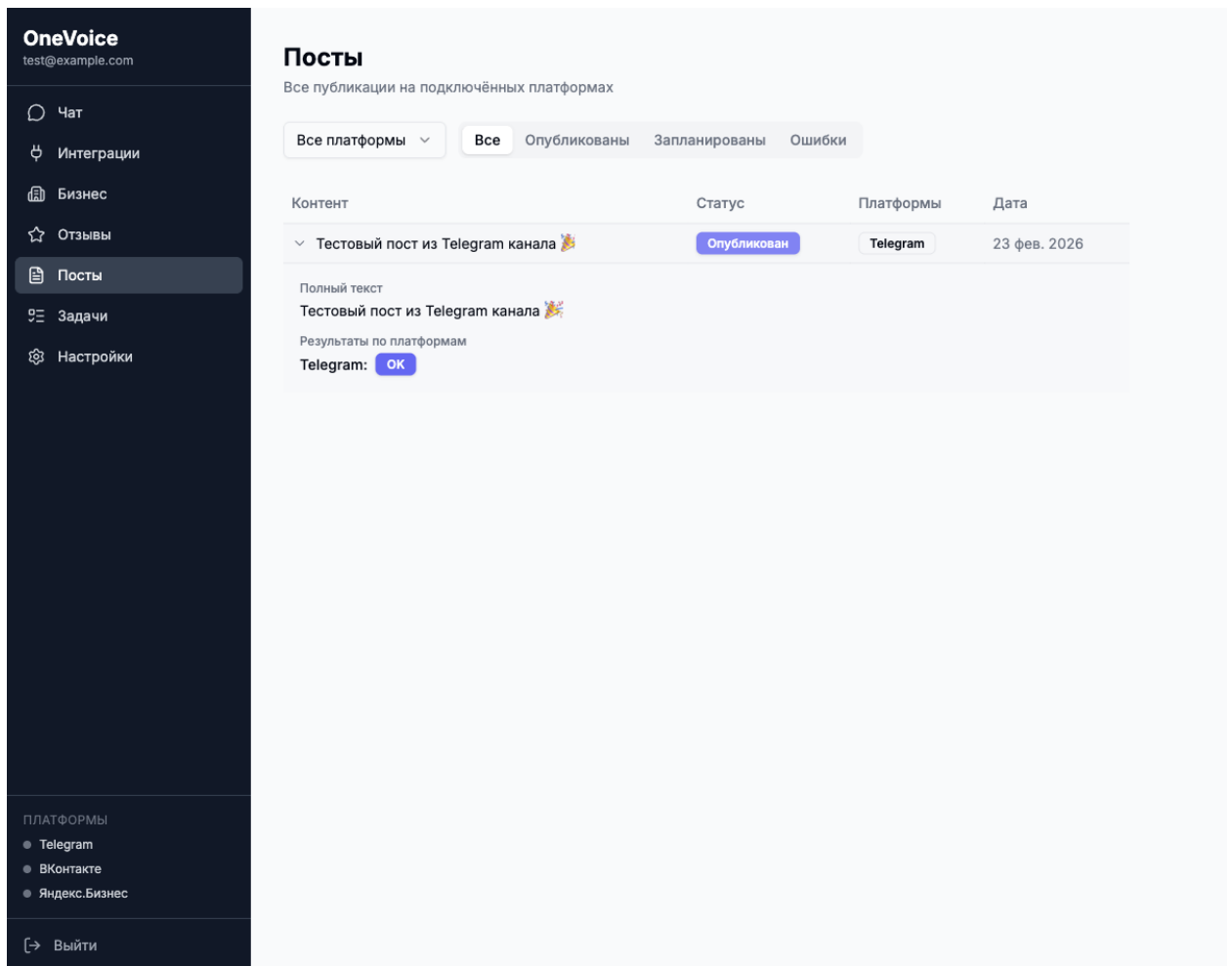


Рисунок 18 – Раскрытая запись публикации с результатами по платформам

### 2.3.3.5 Журнал задач агентов

Страница задач предоставляет мониторинг операций, выполняемых платформенными агентами, с фильтрацией по статусу (в очереди, активные, завершены, ошибки) и платформе (рисунок 19).



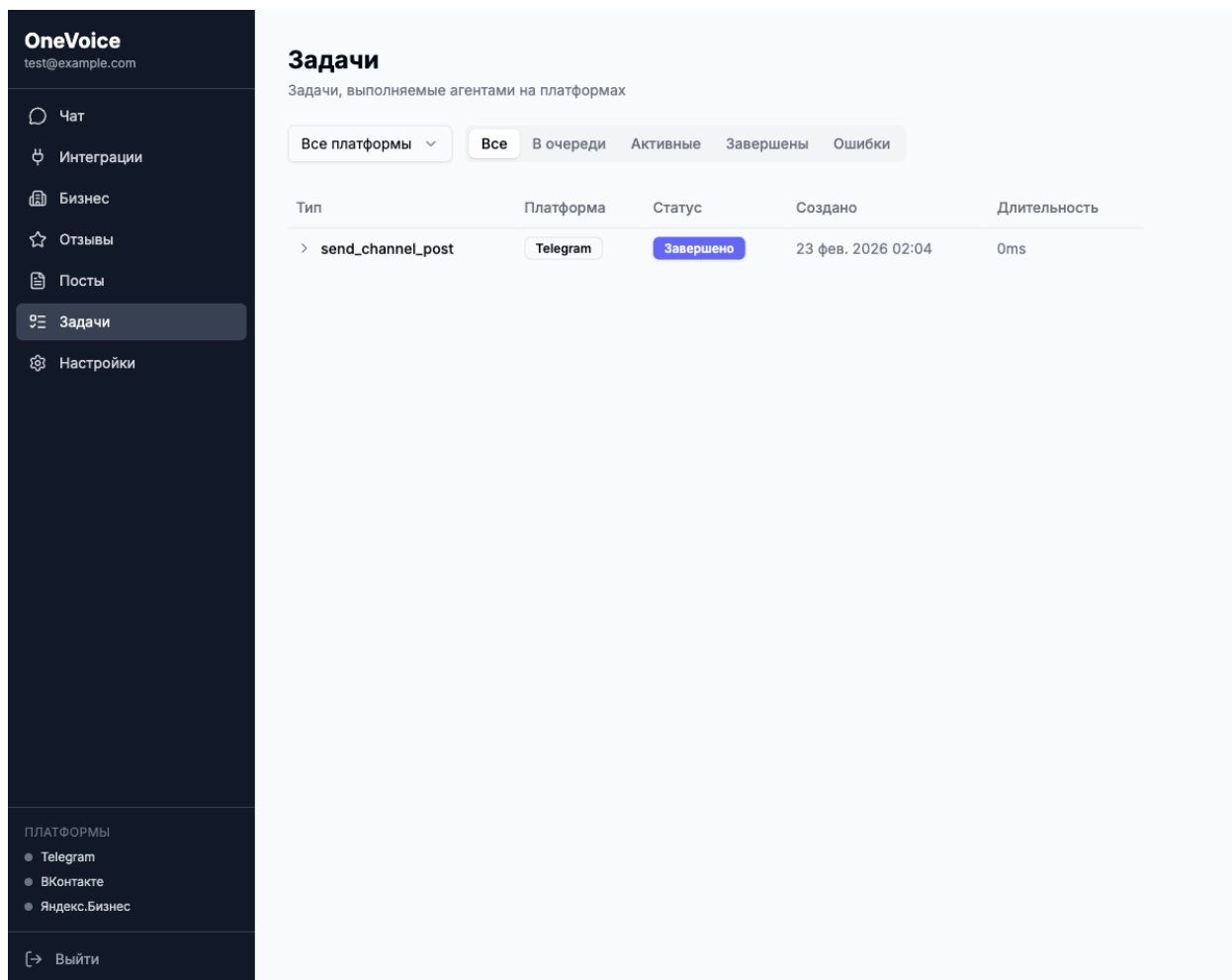


Рисунок 19 – Журнал задач агентов

#### 2.3.4 Управление состоянием

Для управления состоянием клиентского приложения используется двухуровневый подход:

1) **Zustand** — локальное состояние аутентификации (пользователь, токены, статус авторизации) с сохранением состояния в локальном хранилище браузера (localStorage);

2) **TanStack React Query** — серверное состояние (диалоги, интеграции, бизнес-профиль, публикации, отзывы, задачи) с автоматическим кешированием, инвалидацией и повторными запросами.

Потоковая передача ответов LLM реализована через SSE (Server-Sent Events): клиент устанавливает соединение при отправке сообщения и обрабатывает события пяти типов: `text` (текст ответа), `tool_call` (вызов инстру-

мента), `tool_result` (результат инструмента), `done` (завершение), `error` (ошибка).

## 2.4 Средства реализации

Система OneVoice реализована на языках Go (серверная часть) и TypeScript (клиентское приложение). Общий объём исходного кода составляет порядка 26 000 строк: 16 000 строк Go-кода в микросервисах, 4 000 строк в разделяемой библиотеке `pkg` и 6 000 строк TypeScript/TSX в веб-приложении.

Ключевые библиотеки и фреймворки, использованные при разработке, перечислены в таблице 18.

Таблица 18 – Основные библиотеки и фреймворки

| Платформа  | Библиотека                        | Назначение                                       |
|------------|-----------------------------------|--------------------------------------------------|
| Go         | <code>chi v5</code>               | HTTP-маршрутизация и middleware                  |
| Go         | <code>pgx v5</code>               | Драйвер PostgreSQL с поддержкой пула соединений  |
| Go         | <code>nats.go</code>              | Клиент NATS для межсервисной коммуникации        |
| Go         | <code>playwright-go</code>        | Автоматизация браузера для RPA-агента            |
| Go         | <code>golang-jwt v5</code>        | Генерация и валидация JWT-токенов                |
| TypeScript | <code>Next.js 14</code>           | SSR/SSG-фреймворк с App Router                   |
| TypeScript | <code>TanStack Query</code> React | Управление серверным состоянием и кешированием   |
| TypeScript | <code>Zustand</code>              | Управление локальным состоянием (аутентификация) |
| TypeScript | <code>Zod</code>                  | Валидация данных форм и API-ответов              |
| TypeScript | <code>shadcn/ui</code>            | Компонентная библиотека на базе Radix UI         |

Системные требования для развёртывания: Docker Engine 20+, Docker Compose v2, 4 ГБ оперативной памяти, Go 1.24 и Node.js 20 (для локальной разработки). Полный исходный код системы приведён в приложении А.

## 2.5 Инсталляция и настройка

Развёртывание системы OneVoice выполняется с помощью Docker Compose и включает девять контейнеров: шесть микросервисов (API, оркестратор, веб-приложение, Telegram-агент, VK-агент, Yandex.Business-агент) и три инфраструктурных сервиса (PostgreSQL, MongoDB, NATS).

Конфигурация системы осуществляется через переменные окружения, определённые в файле `.env`. Основные параметры включают строку подключения к PostgreSQL (`DATABASE_URL`), адрес MongoDB (`MONGODB_URI`), адрес NATS (`NATS_URL`), ключ шифрования токенов (`ENCRYPTION_KEY`), секрет JWT (`JWT_SECRET`), модель LLM (`LLM_MODEL`) и API-ключи провайдеров.

Процедура развёртывания состоит из следующих шагов:

- 1) Копирование файла `.env.example` в `.env` и заполнение параметров.
- 2) Запуск всех сервисов командой `make up`, которая выполняет `docker compose up -d`.
- 3) Автоматическое применение миграций базы данных при первом запуске API-сервиса.
- 4) Проверка работоспособности по адресу `http://localhost:3000`.

Типовой сценарий начала работы пользователя включает: регистрацию учётной записи, создание профиля бизнеса с заполнением информации и расписания, подключение интеграций с платформами (ввод токена Telegram-бота, данных VK-сообщества, учётных данных Яндекс.Бизнес) и переход к диалогу с ИИ-ассистентом для управления цифровым присутствием.

## 2.6 Выводы по главе 2

В данной главе выполнена техническая реализация системы OneVoice. Спроектирована микросервисная архитектура, включающая шесть компонентов (API-сервис, оркестратор, веб-приложение, три платформенных аген-

та) и разделяемую библиотеку, взаимодействующих через REST, SSE и NATS.

Описаны два ключевых алгоритма: агентный цикл оркестратора, реализующий итеративное взаимодействие с LLM и диспетчеризацию инструментов через протокол A2A, и алгоритм ротации JWT-токенов, обеспечивающий безопасную аутентификацию с автоматическим обновлением сессии.

Модель данных построена на принципе гетерогенного хранения данных: реляционная СУБД PostgreSQL (5 таблиц: пользователи, бизнесы, интеграции, расписания, токены обновления) используется для структурированных данных с ACID-гарантиями, а документоориентированная MongoDB (5 коллекций: диалоги, сообщения, публикации, отзывы, задачи агентов) — для данных с гибкой схемой.

Пользовательский интерфейс реализован как одностраничное веб-приложение на базе Next.js 14 и включает 10 экранных форм. Интерфейс спроектирован с учётом принципов единообразия, адаптивности и минимализма, обеспечивая корректное отображение на настольных компьютерах, планшетах и мобильных устройствах.

Общий объём исходного кода составляет порядка 26 000 строк. Развёртывание системы контейнеризовано с помощью Docker Compose (9 контейнеров), что обеспечивает воспроизводимость среды и упрощает установку.

## ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы были решены все поставленные задачи.

**1) Проведён анализ предметной области и существующих решений.** Исследован процесс управления цифровым присутствием субъектов МСБ, построена модель текущего процесса (AS-IS). Выполнен сравнительный анализ семи существующих решений трёх классов (CRM, SMM, ORM) методом взвешенных критериев, который показал, что ни одно из них не обеспечивает комплексного управления консистентностью данных на разнородных платформах.

**2) Сформулированы функциональные и нефункциональные требования к системе.** Определены семь функциональных требований (управление бизнес-объектами, подключение каналов, контроль консистентности, оркестрация действий, контур коммуникаций, журналирование, ролевой доступ) и восемь нефункциональных требований в привязке к модели качества ISO/IEC 25010:2023. Предложена концептуальная модель данных и описано целевое состояние процесса (TO-BE).

**3) Обоснован выбор технологий и архитектурных решений.** Методом взвешенных критериев обоснован выбор мультиагентной архитектуры с гибридной моделью интеграции (API + RPA), языка Go для серверной части (балл 2,80 из 3,00) и фреймворка Next.js для клиентской части (балл 3,00 из 3,00). Выполнена классификация целевых платформ по типу интеграции.

**4) Спроектирована архитектура мультиагентной системы.** Разработана микросервисная архитектура из шести компонентов (API-сервис, оркестратор, веб-приложение, три платформенных агента) и разделяемой библиотеки, взаимодействующих через REST, SSE и NATS. Описаны протокол Agent-to-Agent (A2A), алгоритм агентного цикла оркестратора и алгоритм ротации JWT-токенов.

5) **Спроектирована модель данных.** Построена модель на принципе гетерогенного хранения: реляционная СУБД PostgreSQL (5 таблиц) для структурированных данных с ACID-гарантиями и документоориентированная MongoDB (5 коллекций) для данных с гибкой схемой.

6) **Реализован прототип программного комплекса.** Общий объём исходного кода составляет порядка 26 000 строк. Реализованы три платформенных агента: Telegram-агент (5 инструментов, Telegram Bot API), VK-агент (9 инструментов, VK API) и Yandex.Business-агент (4 инструмента, браузерная автоматизация Playwright). Пользовательский интерфейс включает 10 экранных форм. Развёртывание контейнеризовано с помощью Docker Compose (9 контейнеров).

7) **Проведено тестирование разработанного решения.** Выполнено функциональное тестирование по критическому пути пользователя для роли владельца бизнеса, подтвердившее работоспособность основных сценариев: регистрация и аутентификация, настройка профиля бизнеса, подключение интеграций, управление данными через ИИ-ассистента, сбор отзывов и публикация контента.

Таким образом, цель работы — разработка программного комплекса на основе мультиагентной архитектуры с гибридной моделью интеграции для управления цифровым присутствием субъектов МСБ — достигнута. Разработанная система позволяет управлять данными на нескольких платформах из единого интерфейса с использованием естественного языка, при этом гибридная модель интеграции обеспечивает работу как с платформами, предоставляющими официальные API, так и с платформами без публичного программного интерфейса.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Google Business Profile Help. Edit your Business Profile. — URL: <https://support.google.com/business/answer/3039617> (дата обращения: 01.02.2026).
2. BrightLocal. Business Listings Trust Report 2021. — URL: <https://www.brightlocal.com/research/business-listings-trust-report/> (дата обращения: 01.02.2026).
3. BrightLocal. Local Business Discovery & Trust Report 2023. — URL: <https://www.brightlocal.com/research/local-business-discovery-trust-report/> (дата обращения: 01.02.2026).
4. РОМИР. Геосервисы и поисковые системы — главные помощники россиян при выборе офлайн-заведений. — 2025-05-15. — URL: <https://romir.ru/studies/romir-geoservisy-i-poiskovye-sistemy--glavnye-pomoshchniki-rossiyan-pri-vybore-oflayn-zavedeniy> (дата обращения: 01.02.2026).
5. BrightLocal. Local Consumer Review Survey 2024. — URL: <https://www.brightlocal.com/research/local-consumer-review-survey-2024/> (дата обращения: 01.02.2026).
6. Аналитический центр НАФИ. Рейтинг факторов, формирующих доверие продавцам в e-commerce. — 2024-06-18. — URL: <https://nafi.ru/analytics/rejting-faktorov-formiruyushchikh-doverie-prodavtsam-v-e-commerce/> (дата обращения: 01.02.2026).
7. Bitrix24. Contact Center (CRM): Free omnichannel contact center solution. — URL: <https://www.bitrix24.com/tools/crm/contact-center.php> (дата обращения: 01.02.2026).
8. Bitrix24. CONTACT CENTER: All-In-One Communication Platform. — URL: [https://www.bitrix24.com/solutions/tool/contact\\_center.php](https://www.bitrix24.com/solutions/tool/contact_center.php) (дата обращения: 01.02.2026).
9. Bitrix24 Helpdesk. Chat tracker. — URL: <https://helpdesk.bitrix24.com/open/14512848/> (дата обращения: 01.02.2026).

10. SendPulse. CRM system (features). — URL: <https://sendpulse.com/features/crm> (дата обращения: 01.02.2026).
11. SendPulse Help Center. Deal pipeline basics. — URL: <https://sendpulse.com/knowledge-base/crm/pipeline> (дата обращения: 01.02.2026).
12. Kommo. Unified inbox. — URL: <https://www.kommo.com/unified-inbox/> (дата обращения: 01.02.2026).
13. Hootsuite. Social Media Calendar and Scheduler (Publishing). — URL: <https://www.hootsuite.com/platform/publishing> (дата обращения: 01.02.2026).
14. Hootsuite Help Center. Bulk schedule your posts. — URL: <https://help.hootsuite.com/hc/en-us/articles/1260804249930-Bulk-schedule-your-posts> (дата обращения: 01.02.2026).
15. Buffer. Social Media Scheduler & Planner (Publish). — URL: <https://buffer.com/publish> (дата обращения: 01.02.2026).
16. Buffer Help Center. Getting started with Buffer's publishing features. — URL: <https://support.buffer.com/article/600-getting-started-with-buffers-publishing-features> (дата обращения: 01.02.2026).
17. SMMplanner. Automated social media posting. — URL: <https://page.smmplanner.io/en/> (дата обращения: 01.02.2026).
18. SMMplanner. Content posting and re-poster for social media and messengers. — URL: <https://smmplanner.com/lang/en/repost> (дата обращения: 01.02.2026).
19. YouScan. AI-powered Social Listening Platform with Visual Analysis (Product). — URL: <https://youscan.io/product/> (дата обращения: 01.02.2026).
20. YouScan. Visual Insights: Next-generation social listening. — URL: <https://youscan.io/visual-insights/> (дата обращения: 01.02.2026).
21. Zhang X. et al. A Survey of Multi-AI Agent Collaboration: Theories, Technologies and Applications // Proceedings of DEAI '25. — 2025. — URL: <https://dl.acm.org/doi/full/10.1145/3745238.3745531> (дата обращения: 01.02.2026).
22. A2A Protocol. Agent2Agent (A2A) Protocol Specification (Release Candidate v1.0). — URL: <https://a2a-protocol.org/latest/specification/> (дата



обращения: 01.02.2026).

23. JSON-RPC Working Group. JSON-RPC 2.0 Specification. — URL: <https://www.jsonrpc.org/specification> (дата обращения: 01.02.2026).

24. Model Context Protocol. Specification (2025-11-25). — URL: <https://modelcontextprotocol.io/specification/2025-11-25> (дата обращения: 01.02.2026).

25. Telegram. Bot API. — URL: <https://core.telegram.org/bots/api> (дата обращения: 01.02.2026).

26. Bhardwaj V. A Systematic Review of Robotic Process Automation in Business Operations: Contemporary Trends and Insights // Journal of Intelligent Systems and Control. — 2023. — Т. 2, № 3. — DOI: 10.56578/jisc020304.

27. El-Gharib N. M., Amyot D. Robotic Process Automation Using Process Mining — A Systematic Literature Review. — arXiv:2204.00751 (2022, revised 2023). — DOI: 10.48550/arXiv.2204.00751.

28. IEEE/ISO/IEC 29148-2018: Systems and software engineering — Life cycle processes — Requirements engineering. — URL: <https://standards.ieee.org/standard/29148-2018.html> (дата обращения: 01.02.2026).

29. ISO/IEC 25010:2023 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. — URL: <https://www.iso.org/standard/78176.html> (дата обращения: 01.02.2026).

30. ISO/IEC 25010 (описание модели качества и характеристик). — URL: <https://iso25000.com/en/iso-25000-standards/iso-25010> (дата обращения: 01.02.2026).

31. OnePageCRM Help Center. Manage sales deals in OnePageCRM with Kanban view. — URL: <https://help.onepagecrm.com/article/642-pipeline-kanban-view> (дата обращения: 01.02.2026).